



UNIVERSIDAD DE BUENOS AIRES
FACULTAD DE CIENCIAS EXACTAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

Escalando la síntesis composicional de controladores mediante una nueva técnica de minimización para branching bisimilarity

Tesis de Licenciatura en Ciencias de la Computación

Cindy Eliana Levi

Director: Hernán Gabriel Gagliardi

Codirector: Sebastian Uchitel  *⟨Creo que vamos a sacar a Sebas para ver si lo ponemos de jurado, dejo esta marca para no olvidarme⟩*

Buenos Aires, 2026

ESCALANDO LA SÍNTESIS COMPOSICIONAL DE CONTROLADORES MEDIANTE UNA NUEVA TÉCNICA DE MINIMIZACIÓN PARA BRANCHING BISIMILARITY

La síntesis composicional de controladores evita la explosión de estados resolviendo el problema de a partes: en cada iteración compone un subconjunto de la planta, sintetiza un controlador parcial, oculta los eventos locales y *minimiza* el resultado antes de devolverlo al sistema. Ese paso de minimización es el que impide que los modelos intermedios vuelvan a crecer, y es también el que domina el costo del método: se lo resuelve calculando *Weak Synthesis Observational Equivalence* (WSOE) con un algoritmo de complejidad $O(nm)$. En esta tesis proponemos reemplazar WSOE por *branching bisimilarity*, para la cual existe un algoritmo de complejidad $O(m \log n)$. Mostramos que el reemplazo no es directo: bajo la hipótesis natural de ocultar todas las acciones locales, *branching* fusiona estados que WSOE mantiene separados, lo que implicaría reducir de más. Demostramos que restringiendo las acciones internas a las acciones locales *no controlables* vale $\sim_B \subseteq \sim_W$, y que la inclusión es estricta. Sobre esa base implementamos el algoritmo dentro de MTSA y lo integramos al flujo composicional.

La evaluación se realizó sobre 2212 instancias reales de minimización capturadas del propio flujo de síntesis. La correctitud se validó contra la implementación de referencia de mCRL2 y contra el resultado de WSOE. En cuanto al rendimiento, la implementación final se mantiene por debajo de los 1000 ms en las instancias más grandes, donde WSOE alcanza los 7600 ms, y en el 83 % de los casos ambas equivalencias producen la misma reducción. El costo del cambio aparece en el uso de memoria.

Palabras claves: Sistemas de Eventos Discretos, Síntesis de Controladores, Síntesis Composicional, Minimización, Branching Bisimilarity, WSOE, Refinamiento de Particiones, LTS.

AGRADECIMIENTOS

A Silvana y Beto, por criarme y acompañarme en cada paso con infinito amor.

A Mariana, Laura, Cecilia y Mariano, porque siempre me hicieron sentir querido, protegido y ayudado.

A la casa de Perón, porque en comunidad transcurrió gran parte de la carrera y en donde Milagros, Ale, Julls, Alicia, Franco, Tito, Martín, Cabeza y Virginia me bancaron las jornadas de estudio y siguieron cada uno de mis exámenes.

A Fernando, Pablo, Santiago y Juan Pablo por el ejercicio incesante de la amistad y por ser un ejemplo a seguir dentro y fuera de la facultad.

A Fabio y a Roni, porque hicimos codo a codo cada examen, final y TP de la carrera.

A Daniela, que en muchos momentos difíciles de la carrera supo ayudarme y convencerme de seguir.

Al Departamento de Computación y toda la gente que lo compone por ser una segunda casa desde hace tantos años. Al LaFHIS y todxs sus integrantes por haberme recibido en este último tramo con los brazos abiertos y haberme apoyado en todo momento.

A mi director de Tesis Sebastian Uchitel y a mi co-director Sebastian Zudaire por la guía y por la inmensa generosidad con la que me acompañaron a lo largo de este trabajo.

A la militancia de Identidad, porque aprender de ellxs que vale la pena organizarse, luchar y trabajar por una facultad y un país mejor es una de las cosas más valiosas que me dejaron estos años. A Juliana y Abril, que me permiten irme del claustro con el inmenso orgullo de haber votado y acompañado a las primeras dos Presidentas Peronistas del CECEN.

A Julia porque construimos juntxs, desde el amor, los días más felices.

Índice general

1..	Introducción	1
2..	Antecedentes	5
2.1.	Sistemas de transiciones etiquetados	5
2.2.	Un ejemplo concreto	7
3..	Branching bisimilarity	11
3.1.	Definición formal	11
3.2.	WSOE: definición formal	11
3.3.	Casos de ejemplo	12
3.4.	Teorema de equivalencia y demostración	15
4..	El algoritmo $O(m \log n)$ de Groote et al.	17
4.1.	Terminología	17
4.2.	Idea general	19
4.3.	Funcionamiento	20
4.4.	La función split	25
4.5.	Detalle de la complejidad	27
5..	Implementación del algoritmo	31
5.1.	Introducción	31
5.2.	Primera implementación del algoritmo	31
5.3.	Primeros tests	35
5.3.1.	Integración con el algoritmo composicional	35
5.4.	Primeras optimizaciones	36
5.4.1.	Cambio de estrategia: dos fases que alternan	36
5.4.2.	Implementación de nuevas estructuras	37
5.5.	Alcanzando la complejidad teórica	37
5.5.1.	Refinable Partitions	38
5.5.2.	Linked Transition Partitions	41
5.5.3.	El método split como corrutinas	43
6..	Validación y experimentación	47
6.1.	Metodología	47
6.2.	Validación de correctitud	48
6.3.	Resultados	48
6.3.1.	Tamaño de la reducción final	48
6.3.2.	Tiempos	49
6.3.3.	Memoria	50
7..	Conclusiones	53
8..	Apéndice	55

1. INTRODUCCIÓN

*Lo que vieron mis ojos fue simultáneo:
lo que transcribiré, sucesivo,
porque el lenguaje lo es.
Algo, sin embargo, recogeré.
—JLB*

La automatización de tareas ha sido a lo largo de la historia un desafío *motivante* para las Ciencias de la Computación. Desde sus inicios se buscó la manera de automatizar las tareas tediosas o repetitivas. Yendo un poco más lejos, la posibilidad de una computadora que *se programe* a sí misma es abordada por distintas ramas de la computación. Entre ellas se encuentran muchas agrupadas dentro del término *Inteligencia Artificial*. En el trabajo que presentamos nos abocaremos al caso particular de programas que generan **Síntesis Automática de Controladores**.

En este área de estudio buscamos desarrollar código que **sintetice** una solución de un determinado problema de forma **automática** partiendo de ciertas *reglas y objetivos* (o pre/post condiciones) que se proveen y es necesario cumplirlos. De esta manera, lo que devuelve nuestro programa es una estrategia que garantiza alcanzar una solución, si es que existe. Por otro lado, en el caso de que dadas las reglas y los objetivos planteados no exista una solución posible, avisa sobre la inexistencia de la misma. En el caso de garantizar la existencia de una solución, a la estrategia que permite alcanzarla se la conoce con el nombre de **Controlador**.

Este tipo de programas tienen múltiples aplicaciones, entre ellas podemos nombrar el control de aviones para demarcar incendios forestales [6], donde a veces incluso es necesario que el programa pueda adaptarse en medio de su ejecución.

El problema de *síntesis* fue estudiado dentro de distintas áreas como: Control de Eventos Discretos [1], Síntesis Reactiva [2] y Planning automático [3]. Si bien este trabajo se basa en resultados que provienen de combinar enfoques de las tres áreas, nos basamos principalmente en Control de Eventos Discretos (Discrete Event Control o DEC).

En este enfoque modelamos el problema utilizando autómatas finitos, también conocidos como máquinas de estados finitas. Un autómata finito, está conformado por estados y transiciones entre ellos. Podemos pensar los *estados* como puntos y las transiciones como flechas de un solo sentido que unen a uno o varios de ellos. A estos puntos los llamamos estados ya que representan un momento particular del dominio que se quiere modelar, por ejemplo en el caso del avión mencionado anteriormente se podría hablar de estados que representen posibilidades como: *avión aterrizado*, *avión sin combustible*, *avión en vuelo*, etc.

Por otro lado, vamos a tomar dentro de nuestro modelo a las transiciones como posibles acciones. En este caso algunas transiciones las podemos controlar y otras estarán fuera de nuestro alcance. Volviendo al ejemplo del avión, una acción **controlable** podría ser *encendido de motor* mientras que la acción *aterrizaje de emergencia* probablemente sea representada en nuestro modelo como una acción **no controlable**.

En el autómata mencionado además existen estados distinguidos. Por un lado tomaremos un estado como *inicial* y será el estado desde el cual empezaremos la exploración.

Además habrá *estados objetivos* que serán los estados a los cuales buscaremos llegar. Buscaremos que el sistema no solo pueda alcanzarlos, sino que sea capaz de hacerlo infinitas veces y de manera segura.

El valor de este enfoque y de un algoritmo que permita realizar síntesis es la generalización a distintos problemas. El autómata descrito puede modelar un avión como el caso mencionado, o el funcionamiento de una agencia de viajes como se verá en otro ejemplo o podrá modelar el funcionamiento de diversos problemas complejos. De todas formas, nuestro algoritmo podrá generar un **controlador** sin necesitar ningún cambio específico. Para lograr esto trabajará un experto en el área, quien decide las reglas y objetivos que conformarán el autómata a analizar, por ejemplo partir de un avión en un hangar e intentando llegar a un estado objetivo que describa a un avión aterrizado luego de formar un patrón en el cielo. Pero como podemos ver, este experto no debe preocuparse por como se sintetiza el controlador que resuelve el autómata que modela su problema, esa parte está garantizada por el algoritmo de síntesis que genera el controlador.

Por último cabe destacar que el modelado de un sistema complejo resulta prácticamente imposible utilizando un solo autómata. Por esta razón es que se suelen modelar distintas partes del problema original como autómatas independientes y luego se componen para formar el modelo completo del sistema. Esta composición a la que nos referimos debe respetar ciertas reglas y garantiza que un estado del sistema compuesto representa la combinación de estados en que está cada componente al finalizar.

Tradicionalmente la forma de resolver el problema consiste en tomar estos autómatas independientes, realizar la composición y obtener el autómata completo para que sea analizado por el programa para sintetizar el controlador. Sin embargo, cuando el problema a resolver escala en tamaño de estados y transiciones de forma abrupta el autómata compuesto se vuelve demasiado grande, haciendo imposible su análisis y en ocasiones resultando imposible también el proceso de composición. Este fenómeno, conocido como *explosión de estados*, es el obstáculo central para que la síntesis de controladores sea aplicable a problemas de tamaño realista.

Frente a la explosión de estados se han propuesto distintas estrategias. Una de ellas es la exploración *on-the-fly*, en donde la composición se construye a medida que se van explorando transiciones, evitando en la medida de lo posible componer todo el modelo; este enfoque fue presentado en la tesis doctoral de Daniel Ciolek [7] y profundizado luego en el trabajo de Matías Durán y Florencia Zanollo [8, 9].

La estrategia sobre la que se apoya esta tesis es distinta, y es la **síntesis composicional** [10]. La idea es no componer nunca el sistema completo, sino trabajar de manera incremental: se selecciona un subconjunto de los componentes, se lo compone, se sintetiza un controlador *parcial* sólo para esa subplanta, y se la reemplaza por una versión más chica de sí misma. Esa versión más chica se obtiene en dos pasos: primero se ocultan los eventos que la subplanta no comparte con el resto del sistema (denotados eventos locales, que al ser internos, ya no pueden influir en el comportamiento de los demás componentes), y después se **minimiza** el modelo resultante, fusionando aquellos estados que pasaron a ser indistinguibles. El resultado se devuelve al conjunto de componentes y el procedimiento se repite.

De esta manera, el modelo que efectivamente hay que analizar en cada paso se mantiene acotado, y la solución final no es un controlador monolítico sino un conjunto de controladores parciales que corren en paralelo. Este enfoque permite resolver problemas de control hasta 1000 veces más grandes que los que resuelve el enfoque monolítico [10].

El paso de minimización no es un detalle de implementación sino que es el paso que se ejecuta una y otra vez, una vez por cada iteración del procedimiento. Los autores de [10] identifican a este paso como el cuello de botella del proceso de síntesis composicional.

Para que la minimización sea correcta se utiliza una *noción de equivalencia* que preserve las soluciones del problema de control. La noción que usa el algoritmo composicional es *Weak Synthesis Observational Equivalence* (WSOE), tomada del marco de control supervisado de Mohajerani, Malik y Fabian [11].

El problema es su costo. El algoritmo que calcula WSOE tiene complejidad $O(nm)$ sobre el modelo a minimizar, con n su cantidad de estados y m la de transiciones. Como se lo invoca una vez por iteración y sobre modelos que pueden ser grandes, ese costo termina dominando el tiempo total de la síntesis composicional. Mejorar la minimización es, entonces, mejorar la escalabilidad de todo el método.

Existe una noción de equivalencia muy estudiada en la literatura de semántica de procesos que es cercana a WSOE, *branching bisimilarity*, y para la cual Groote, Jansen, Keiren y Wijs [20] presentaron un algoritmo considerablemente más eficiente, que calcula las particiones en tiempo $O(m \log n)$.

La propuesta de esta tesis es reemplazar WSOE por *branching bisimilarity* como técnica de minimización dentro del flujo de síntesis composicional, e implementar el algoritmo $O(m \log n)$ de Groote et al. dentro de la herramienta MTSA [22].

En el Capítulo 2 presentamos los antecedentes y definiciones necesarias alrededor del problema de síntesis composicional de controladores. En el Capítulo 3 introducimos *branching bisimilarity*, damos la definición formal de WSOE y demostramos el teorema de equivalencia entre ambas. En el Capítulo 4 presentamos el algoritmo $O(m \log n)$ de Groote et al. [20] que calcula *branching bisimilarity* de forma eficiente. Luego, en el Capítulo 5, detallamos su adaptación e implementación dentro del flujo composicional de MTSA [22]. En el Capítulo 6 presentamos la validación y la experimentación, comparando la minimización por WSOE contra *branching bisimilarity* en tiempo, memoria y tamaño de los LTS minimizados. Finalmente, en el Capítulo 7 exponemos las conclusiones y el trabajo futuro.

2. ANTECEDENTES

En este capítulo agregaremos algunas definiciones y explicaciones necesarias para poder avanzar con la definición y análisis formal del problema composicional de síntesis de controladores.

2.1. Sistemas de transiciones etiquetados

Modelamos sistemas de eventos discretos componiendo *sistemas de transiciones etiquetados* (*Labelled Transition Systems* o LTS). Un LTS es la formalización de la noción intuitiva de autómatata que presentamos en el Capítulo 1.

Definición 2.1.1 (LTS). Un *sistema de transiciones etiquetado* (LTS) es una tupla $M = (S, \Sigma, \rightarrow, \bar{s})$ donde S es un *conjunto finito de estados*; Σ es un *conjunto finito de etiquetas de eventos*; $\rightarrow \subseteq S \times \Sigma \times S$ es la *relación de transiciones*; y $\bar{s} \in S$ es el *estado inicial*.

Notamos $(s, \ell, s') \in \rightarrow$ como $s \xrightarrow{\ell} s'$. Esta es exactamente la estructura sobre la que se definen las dos nociones de equivalencia del Capítulo 3 y sobre la que opera el algoritmo del Capítulo 4.

Notación. Usamos Σ^* para el conjunto de secuencias finitas de etiquetas de Σ , a las que llamamos *trazas*. Decimos que $\ell \in \Sigma$ está *habilitado* en s si existe s' tal que $s \xrightarrow{\ell} s'$, y notamos $\rightarrow(s)$ al conjunto de eventos habilitados en s . Un LTS es *determinístico* si ningún estado tiene dos transiciones salientes con el mismo evento, y es *libre de deadlock* si todo estado tiene al menos una transición saliente. Extendemos $\rightarrow$ a trazas de la manera usual, asumiendo que vale para la traza vacía: $s \xrightarrow{\epsilon} s$. Decimos que una secuencia w de etiquetas de Σ , finita o infinita, es una *traza de M* si existe s' tal que $\bar{s} \xrightarrow{w} s'$.

Tal como se anticipó en el Capítulo 1, un sistema complejo rara vez se modela con un único LTS: se describen sus partes por separado y luego se las combina. La operación que las combina es la composición paralela, que sincroniza los componentes en los eventos que comparten y los deja evolucionar de manera independiente en el resto.

Definición 2.1.2 (Composición paralela). Sean $M_i = (S_i, \Sigma_i, \rightarrow_{M_i}, \bar{s}_i)$ con $i \in \{1, 2\}$ dos LTS. Su *composición paralela* es el LTS $M_1 \parallel M_2 = (S_1 \times S_2, \Sigma_1 \cup \Sigma_2, \rightarrow_{M_1 \parallel M_2}, \langle \bar{s}_1, \bar{s}_2 \rangle)$, donde $\rightarrow_{M_1 \parallel M_2}$ es la menor relación que satisface las siguientes reglas:

$$\frac{s_1 \xrightarrow{\ell}_{M_1} s'_1}{\langle s_1, s_2 \rangle \xrightarrow{\ell}_{M_1 \parallel M_2} \langle s'_1, s_2 \rangle} \ell \in \Sigma_1 \setminus \Sigma_2 \qquad \frac{s_2 \xrightarrow{\ell}_{M_2} s'_2}{\langle s_1, s_2 \rangle \xrightarrow{\ell}_{M_1 \parallel M_2} \langle s_1, s'_2 \rangle} \ell \in \Sigma_2 \setminus \Sigma_1$$

$$\frac{s_1 \xrightarrow{\ell}_{M_1} s'_1 \quad s_2 \xrightarrow{\ell}_{M_2} s'_2}{\langle s_1, s_2 \rangle \xrightarrow{\ell}_{M_1 \parallel M_2} \langle s'_1, s'_2 \rangle} \ell \in \Sigma_1 \cap \Sigma_2$$

La composición paralela es asociativa y conmutativa. Si $\mathcal{M} = \{M_1, \dots, M_n\}$ es un conjunto de LTS, abusamos de la notación y escribimos $\mathcal{M}$ también para referirnos a $M_1 \parallel \dots \parallel M_n$; llamamos *planta* a ese sistema compuesto y *subplanta* a la composición de un subconjunto de $\mathcal{M}$.

Hay dos características de la composición paralela que conviene destacar. La primera es que la tercera regla es la que realiza la sincronización entre componentes: un evento compartido sólo puede ocurrir si todos los componentes que lo tienen en su alfabeto pueden ejecutarlo simultáneamente. La segunda, y la que motiva todo este trabajo, es que el número de estados de $M_1 \parallel \dots \parallel M_n$ puede crecer **exponencialmente** con respecto al número de autómatas a componer, ya que su espacio de estados es un subconjunto del producto cartesiano $S_1 \times \dots \times S_n$. Este fenómeno se conoce como *explosión de estados* y es lo que vuelve intratable el enfoque monolítico de construir la composición completa antes de analizarla.

Cuando trabajamos con una subplanta M dentro de un sistema modular más grande, no todos sus eventos tienen el mismo estatus. Algunos son la interfaz de M con el resto del sistema: son los que aparecen también en el alfabeto de otros componentes y, por la tercera regla de la Definición 2.1.2, son los puntos en los que M se sincroniza. Otros son internos a M : ningún otro componente los tiene en su alfabeto y, por lo tanto, ninguna decisión del resto del sistema depende de ellos. Esta distinción es la que habilita a abstraer, y por lo tanto la que habilita a minimizar.

Particionamos entonces el alfabeto de un LTS de dos maneras distintas y **ortogonales** entre sí, y es importante no confundirlas porque el resto de la tesis las usa simultáneamente.

La primera partición separa según la **visibilidad**:

$$\Sigma = \Omega \dot{\cup} \Upsilon$$

donde Ω es el conjunto de eventos *compartidos* (u *observables*) —los que M comparte con algún otro componente del sistema o los que aparecen en el objetivo de control— y Υ es el conjunto de eventos *locales*, propios de M . A Υ lo llamamos también el *alfabeto local* de M .

La segunda partición separa según la **controlabilidad**, tal como se introdujo en el Capítulo 1:

$$\Sigma = \Sigma_c \dot{\cup} \Sigma_u$$

donde Σ_c es el conjunto de eventos *controlables*, aquellos que el controlador puede deshabilitar, y $\Sigma_u = \Sigma \setminus \Sigma_c$ es el conjunto de eventos *no controlables*, que el controlador puede observar pero no impedir.

Al cruzar ambas particiones obtenemos cuatro clases de eventos. Dos de ellas van a ser centrales en el Capítulo 3 y merecen nombre propio:

$$\Upsilon_c = \Upsilon \cap \Sigma_c \quad \Upsilon_u = \Upsilon \cap \Sigma_u$$

es decir, las acciones locales controlables y las acciones locales no controlables, respectivamente. Notar que $\Upsilon = \Upsilon_c \dot{\cup} \Upsilon_u$.

Para hablar de lo que un observador externo percibe de una traza, usamos la proyección natural.

Definición 2.1.3 (Proyección natural). Dado $\Omega \subseteq \Sigma$, definimos la *proyección natural* $P_\Omega : \Sigma^* \mapsto \Omega^*$ como la operación que borra de una traza $\pi \in \Sigma^*$ todos los eventos que no pertenecen a Ω .

Es decir, $P_\Omega(\pi)$ es lo que queda de π una vez que se descartan los eventos locales. En particular, si $\ell \in \Omega$ entonces $P_\Omega(\ell) = \ell$, y si $\ell \in \Upsilon$ entonces $P_\Omega(\ell) = \epsilon$.

Todo el trabajo de esta tesis se realizó sobre MTSA (*Modal Transition System Analyzer*) [22], la herramienta desarrollada en el LaFHIS (Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software) de esta facultad. MTSA es un entorno escrito en Java para el modelado, análisis y verificación de sistemas concurrentes, que soporta además de manera nativa la síntesis de controladores para plantas modulares descritas como composición de LTS.

2.2. Un ejemplo concreto

Para ejemplificar y tener una noción acerca del tamaño que puede alcanzar el problema de composición, presentaremos una versión simplificada del problema **TravelAgency**.

En este problema se busca armar una agencia de venta online de paquetes vacacionales. El objetivo es que la misma orqueste los servicios (alquiler de auto, hotel, pasaje de avión, etc.) de forma automática de manera de obtener un paquete de vacaciones completo de ser posible, evitando pagar por paquetes incompletos.

Para cada servicio que se desea sub-contratar se consulta (query) si ese servicio está disponible. En caso de tener éxito y confirmar la disponibilidad, se espera a confirmar el resto para comprar todos al mismo tiempo. En caso de que algún otro servicio no esté disponible, se cancela la compra y se vuelve al estado inicial, lo cual no implica un gasto.

Este problema, descrito por ahora en términos generales, puede escalar de forma muy rápida si se incrementa la cantidad de servicios a contratar o la cantidad de pasos para reservar cada servicio, por lo que para este ejemplo ilustrativo asumimos que no hacen falta múltiples pasos para reservar un servicio, y solo mostramos el problema con 1 y 2 sub-servicios.

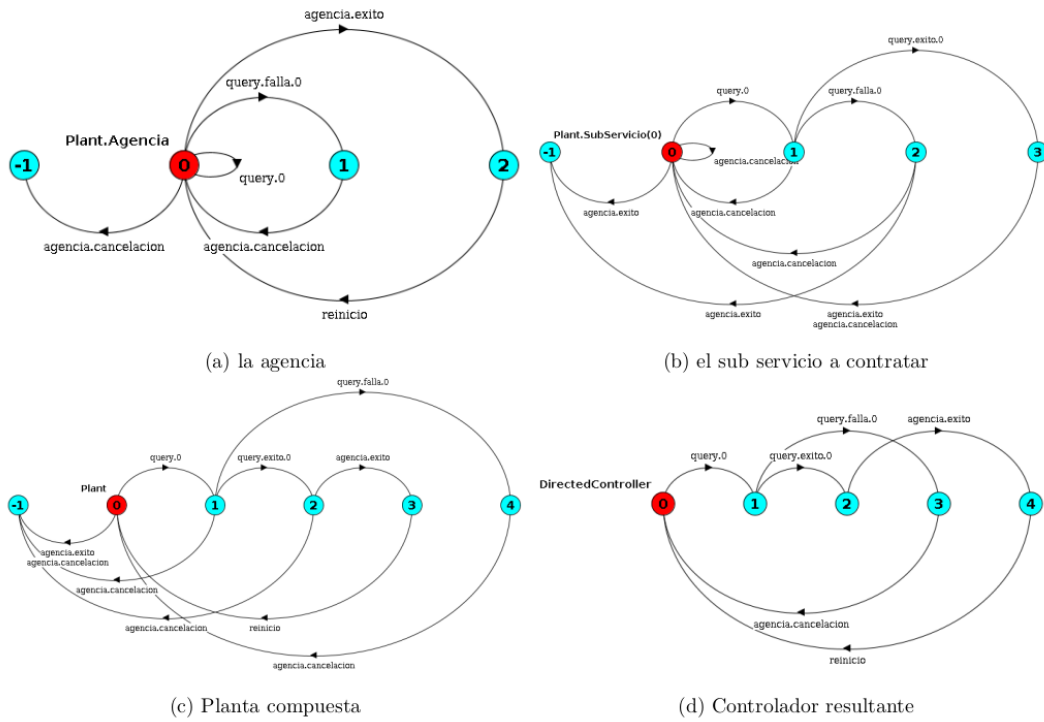


Fig. 2.1: Ejemplo del problema Travel Agency (TA) con un solo sub-servicio

En el ejemplo, la figura 2.1a es la representación de la agencia que busca confirmar la compra, el estado 2 es nuestro estado objetivo, al que queremos alcanzar infinitas veces. En cambio, al detectar algún *query.falla*, se mueve al estado 1, señalando que hubo un problema y solo permite cancelar la compra de todos los otros servicios consultados.

El componente del *subservicio* (figura 2.1b) también tiene el evento *agencia.exito*, pero debe previamente haber recibido la consulta y confirmado su disponibilidad tomando el evento *query.exito.0*.

En caso de no ser posible la reserva, solo permite la cancelación de la compra, yendo del estado 2 al 0; si se quisiera confirmar la compra (*agencia.exito*) se llega al **estado -1**, el estado de error reservado para las situaciones que el controlador debe evitar.

Podemos observar la representación del sistema complejo en la sincronización que se ve reflejada en la figura 2.1c, que muestra la planta que representa al sistema con los dos componentes previamente descritos.

Finalmente, 2.1d muestra un posible controlador para este problema. Destacamos que todos los eventos controlables que llevaban al estado -1, no son una posibilidad permitida por el controlador.

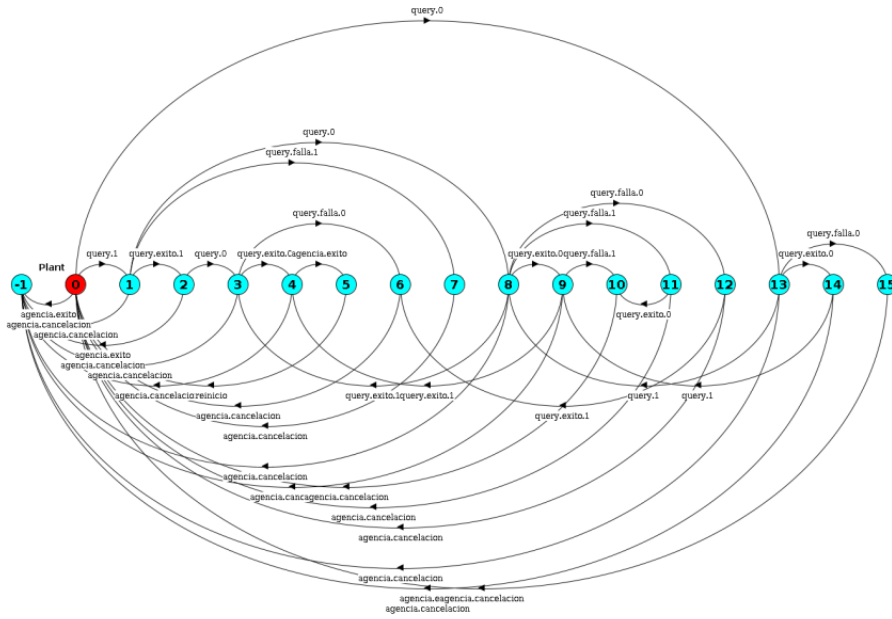


Fig. 2.2: Planta compuesta para 2 sub-servicios

Si analizamos el tamaño que adquiere la planta compuesta expresada en la figura 2.2 podemos notar que es un tipo de problema que escala rápidamente cuando se incrementan los autómatas que se van a componer. También podemos ver que el controlador posee un subconjunto de estados y transiciones de la planta compuesta total.

Este ejemplo ilustra por qué el enfoque composicional evita construir la planta completa: en lugar de componer los dos sub-servicios y la agencia para después analizar el resultado, la síntesis composicional resolvería primero la subplanta formada por la agencia y un sub-servicio, la reemplazaría por su versión controlada y minimizada y recién entonces la compondría con el sub-servicio restante. El tamaño del modelo que efectivamente hay que analizar en cada paso depende entonces, de manera directa, de cuánto reduzca la

minimización.

3. BRANCHING BISIMILARITY

En este capítulo introduciremos las definiciones de Branching bisimilarity y de Weak Synthesis Observational Equivalence (WSOE) para utilizarlas como fundamentos a la hora de minimizar un LTS. Con el objetivo de minimizar un LTS en el contexto de la síntesis composicional, utilizamos una noción de equivalencia para poder aplicarla a la hora de procesar los estados y definir de qué manera decidir si son equivalentes o no. En principio, en el algoritmo de síntesis composicional presentado por Gagliardi et al. [10] se utiliza la definición de WSOE. Tal como se explicó anteriormente, esta tesis consiste en analizar el reemplazo del algoritmo que utiliza la definición de WSOE por el algoritmo de Groote et al. [20] que utiliza la definición de Branching bisimilarity. Para ello, este capítulo introducirá de forma teórica ambas equivalencias y presentará un teorema de cómo se relacionan entre ellas.

3.1. Definición formal

La Definición 3.1.1 introduce la noción de equivalencia denominada Branching bisimulation, propuesta originalmente por van Glabbeek y Weijland [15]. A diferencia de la *observational equivalence* (o *weak bisimulation*) de Milner [14], branching bisimilarity preserva además la estructura de ramificación del LTS, es decir, exige que los estados intermedios que atraviesa un camino de acciones internas sigan siendo equivalentes al estado que se está simulando. Adoptamos la formulación de Groote et al. [20], que es la que utiliza el algoritmo del Capítulo 4.

Dada una relación de equivalencia R , una transición $x_1 \xrightarrow{a} y_1$ se denomina *inerte* si $a = \tau$ y $x_1 R y_1$.

El conjunto de acciones a tal que $a = \tau$ forma parte del conjunto de datos que se deben proveer al comenzar el proceso de minimización.

Definición 3.1.1 (Branching bisimilarity [20]). Sea $M = (S, \Sigma, \rightarrow, \hat{s})$ un LTS. Definimos una relación $R \subseteq S \times S$ como una relación de *branching bisimulation* si es simétrica y para todo $x_1, x_2 \in S$ tal que $x_1 R x_2$ y para toda transición $x_1 \xrightarrow{a} y_1$ tenemos:

1. $a = \tau$ y $y_1 R x_2$, o
2. existe una secuencia $x_2 \xrightarrow{\tau} \dots \xrightarrow{\tau} x'_2 \xrightarrow{a} y_2$ tal que $x_1 R x'_2$ y $y_1 R y_2$.

Los estados x_1 y x_2 son *branching bisimilar*, denotado como $x_1 \sim_B x_2$: si existe una relación de *branching bisimulation* R tal que $x_1 R x_2$.

3.2. WSOE: definición formal

La Definición 3.2.1 introduce la noción de equivalencia denominada Weak Synthesis Observational Equivalence (WSOE) en el contexto de síntesis de controladores propuesto por Gagliardi et al. [10].

Definición 3.2.1 (Weak Synthesis Observational Equivalence [11]). Sea $M = (S, \Sigma, \rightarrow, \hat{s})$ un LTS y sea Σ_c el conjunto de eventos controlables y sea Σ_u el conjunto de eventos

no-controlables. Una relación de equivalencia $\sim_W \subseteq S \times S$ es del tipo *weak synthesis observational equivalence* con M respecto al conjunto de etiquetas Υ_c y Υ_u si las siguientes condiciones valen para todo $x_1, x_2 \in S$ tal que $x_1 \sim_W x_2$:

1. si $x_1 \xrightarrow{u} y_1$ para $u \in \Sigma_u$, entonces existen $\pi_1, \pi_2 \in \Upsilon_u^*$ tales que $x_2 \xrightarrow{\pi_1 P_\Omega(u) \pi_2} y_2$ y $y_1 \sim_W y_2$
2. si $x_1 \xrightarrow{c} y_1$ para $c \in \Sigma_c$, entonces existe un camino $x_2 = x_2^0 \xrightarrow{\tau_1} \dots \xrightarrow{\tau_n} x_2^n \xrightarrow{P_\Omega(c)} y_2$ tal que $y_1 \sim_W y_2$ y $\tau_1, \dots, \tau_n \in \Upsilon$, y si $\tau_{i \in \{1..n\}} \in \Sigma_c$ entonces $x_1 \sim_W x_2^i$

Recordemos que P_Ω es la proyección natural de la Definición 2.1.3, que borra de una traza todos los eventos locales.

3.3. Casos de ejemplo

En esta sección intentaremos comprender la intuición de las definiciones de Branching Bisimilarity y WSOE primero analizando ejemplos más chicos y luego ampliándolos. Al comenzar la tesis, adoptamos la hipótesis más simple posible, es decir que las acciones internas τ son todas las acciones locales. Veamos a través de algunos ejemplos concretos si esta hipótesis resulta correcta.

Ejemplo 1 Consideremos el LTS de la Figura 3.1. La acción t es local y, bajo la hipótesis del caso base, la tomamos como transición interna τ ; las acciones a y b son observables y *no controlables* ($a, b \in \Sigma_u$).

La transición $s_1 \xrightarrow{t} s_2$ es inerte, ya que s_1 y s_2 tienen el mismo comportamiento observable y ambos terminan ejecutando b . Por lo tanto, branching los fusiona: $s_1 \sim_B s_2$ por la primera cláusula de la Definición 3.1.1 (τ inerte).

WSOE llega a la misma conclusión. La transición visible clave es $s_2 \xrightarrow{b} s_3$ con $b \in \Sigma_u$, donde se aplica la primera cláusula de la Definición 3.2.1, ya que b es no controlable. Desde s_1 , b se puede simular con $s_1 \xrightarrow{\tau} s_2 \xrightarrow{b} s_3$. Por lo tanto, $s_1 \sim_W s_2$ usando la cláusula 1 y también $s_1 \sim_B s_2$.

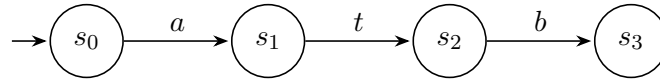


Fig. 3.1: Ejemplo 1. Acción local t (inerte τ para Branching Bisimilarity). a y b son observables no controlables. Vale $s_1 \sim_W s_2$ y $s_1 \sim_B s_2$.

Ejemplo 2 Veamos ahora un ejemplo que utilice la segunda cláusula de la Definición 3.2.1, introduciendo una acción controlable. En el LTS de la Figura 3.2, t es local (τ para Branching Bisimilarity), a es observable no controlable ($a \in \Sigma_u$), y c es una acción observable controlable ($c \in \Sigma_c$). Para ambas equivalencias, los estados s_3 y s_4 son trivialmente equivalentes.

Observemos las equivalencias para los estados s_1 y s_2 . Consideremos la definición de Branching Bisimilarity. Desde s_1 , se puede realizar la acción c tanto directamente ($s_1 \xrightarrow{c} s_4$) como después de la transición interna t ($s_1 \xrightarrow{t} s_2 \xrightarrow{c} s_3$). Podemos aplicar la segunda cláusula de la Definición 3.1.1, ya que ahora necesitamos un camino de τ para

luego simular la acción visible c . Como t es inerte y ambos caminos ofrecen la misma acción controlable c hacia estados equivalentes, branching fusiona $s_1 \sim_B s_2$.

Observemos ahora cómo se comporta la definición de WSOE para este caso. Como $c \in \Sigma_c$, se aplica la segunda cláusula de la Definición 3.2.1. Desde s_2 , la transición $s_1 \xrightarrow{c} s_4$ se puede simular con el camino $s_2 \xrightarrow{P_\Omega(c)} s_3$, con $s_3 \sim_W s_4$. Por lo tanto, $s_1 \sim_W s_2$.

Pudimos ver entonces casos simples que aplican ambas cláusulas de las dos definiciones de equivalencia.

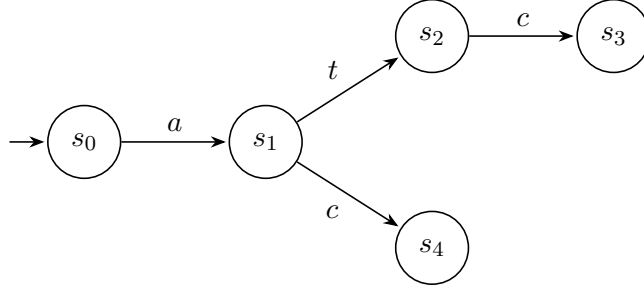


Fig. 3.2: Ejemplo 2. t es local (τ para Branching Bisimilarity), a es observable no controlable ($a \in \Sigma_u$), y c es una acción observable controlable ($c \in \Sigma_c$). Vale que $s_1 \sim_W s_2$ y también $s_1 \sim_B s_2$

Hasta acá la hipótesis “ τ = todas las acciones locales” parece correcta. Veamos un ejemplo más complejo que pudimos descubrir a raíz de diversas experimentaciones realizadas.

Ejemplo 3 La Figura 3.3 ilustra un nuevo ejemplo, donde c_L es controlable y local, c_2 es controlable pero no local, y u es no controlable ($u \in \Sigma_u$). Los estados s_0 y s_1 comparten el comportamiento visible c_2 , y además $s_0 \xrightarrow{c_L} s_1$ y $s_1 \xrightarrow{u} s_2$. Bajo la hipótesis del caso base, al ser c_L local, se la oculta como τ . Podemos nuevamente observar que s_3 y s_4 son trivialmente equivalentes, por lo que vale tanto $s_3 \sim_B s_4$ como $s_3 \sim_W s_4$.

Observemos cómo se comporta la definición de Branching Bisimilarity. Tomemos τ = todas las locales tal como se había planteado. En ese caso, como s_0 y s_1 pueden realizar c_2 hacia estados equivalentes, y como c_L es τ , la acción u de s_1 se puede alcanzar desde s_0 atravesando esa τ , entonces vale que $s_0 \sim_B s_1$.

Veamos cómo se comporta la definición de WSOE. El punto está en la transición no controlable $s_1 \xrightarrow{u} s_2$, ya que para que s_0 la simule, la primera cláusula de la Definición 3.2.1 exige un camino $s_0 \xrightarrow{\tau} \dots \xrightarrow{u} \dots$ cuyas transiciones internas sean no controlables ($\tau \in \Upsilon_u$). Pero la única forma posible desde s_0 de alcanzar la u es a través de c_L , que es *controlable* y por lo tanto no es un τ admisible para WSOE. No existe entonces ningún camino válido, y resulta $s_0 \not\sim_W s_1$.

En este ejemplo entonces, se nos presenta una contradicción, ya que acá branching reduce más que WSOE, fusionando un par que WSOE mantiene separado, porque trata la controlable local c_L como una τ cualquiera. Esto es algo que no queremos que ocurra, ya que necesitamos mantener la lógica de minimización utilizada para el contexto del procesamiento de síntesis de controladores, por lo que en ningún caso debería poder suceder que dos estados sean minimizados con la Definición 3.1.1 de Branching Bisimilarity y no lo sean para la Definición 3.2.1 de WSOE.

El Ejemplo 3 muestra que para poder cumplir con la condición de que Branching no

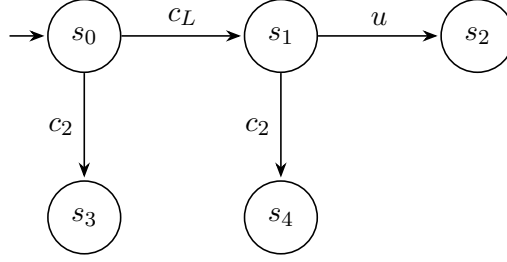


Fig. 3.3: Ejemplo 3. c_L es controlable y local, c_2 es observable controlable y u es observable no controlable.

minimice más que WSOE, no podemos definir al conjunto de transiciones denominadas τ como “todas las locales”. Es decir, el conjunto de acciones τ no puede ser equivalente a $\Upsilon_u \cup \Upsilon_c$. El problema son las acciones controlables locales, ya que podríamos incurrir en casos como este ejemplo. Por eso, tuvimos que redefinir la forma de tomar a las acciones τ en la definición de Branching, restringiéndolas a las **acciones locales no controlables**. Es decir,

$$\tau = \Upsilon \setminus \Sigma_c = \Upsilon_u.$$

Con esta definición c_L vuelve a ser visible en el Ejemplo 3, por lo que la transición $s_0 \xrightarrow{c_L} s_1$ deja de ser inerte y branching pasa a distinguir s_0 de s_1 igual que WSOE.

Esta actualización en la forma de definir a las acciones inertes no cambia el algoritmo de minimización detallado por Groote et al. [20], sino que solo recibe como conjunto de acciones internas el alfabeto local intersecado con las no controlables. Veremos más adelante, en la Sección 3.4, que al definir las acciones inertes de esta manera la definición de WSOE siempre minimiza al menos tanto como la definición de Branching Bisimilarity.

Ejemplo 4 Veamos ahora un ejemplo concreto, adaptado del trabajo de van Glabbeek y Weijland [15], en el que WSOE y Branching Bisimilarity difieren. El LTS se ilustra en la Figura 3.4.

Las acciones τ son locales no controlables ($\tau \in \Upsilon_u$). Los estados $s_4, s_5, s_6, s_{10}, s_{11}, s_{12}$ son trivialmente equivalentes entre sí, tanto para branching como para WSOE.

Queremos analizar cómo se comportan ambas equivalencias al preguntarnos si los estados s_2 y s_8 son equivalentes.

¿ $s_2 \sim_B s_8$? Sigamos la Definición 3.1.1 y observemos qué transiciones puede realizar s_2 . Por un lado, $s_2 \xrightarrow{d_1} s_4$ y $s_8 \xrightarrow{d_1} s_{10}$, entonces vale la segunda cláusula de la definición de Branching y esta transición cumple, pero la definición pide observar todas las acciones posibles. Tomemos la otra transición saliente del estado, $s_2 \xrightarrow{\tau} s_3$. Aquí podemos aplicar la cláusula 1 de la definición, que nos exige observar si $s_3 \sim_B s_8$. Analicemos las transiciones de s_8 , en primera instancia $s_8 \xrightarrow{d_1} s_{10}$. Es claro que s_3 no puede alcanzar a realizar esta acción, ya que no existe ninguna secuencia posible desde el estado que la alcance. Luego, $s_3 \not\sim_B s_8$ y por lo tanto, $s_2 \not\sim_B s_8$.

¿ $s_2 \sim_W s_8$? Pasemos a observar cómo se comporta la definición de WSOE. Para realizar la prueba completa, asumamos a las acciones observables como acciones controlables, ya que así establecemos el caso más estricto de WSOE.

Supongamos entonces $u_1, u_2, a, b, d_1, d_2 \in \Sigma_c$. Estaremos entonces observando la cláusula 2 de WSOE. Analicemos las transiciones de s_2 . En primer lugar, $s_2 \xrightarrow{d_1} s_4$. Para este

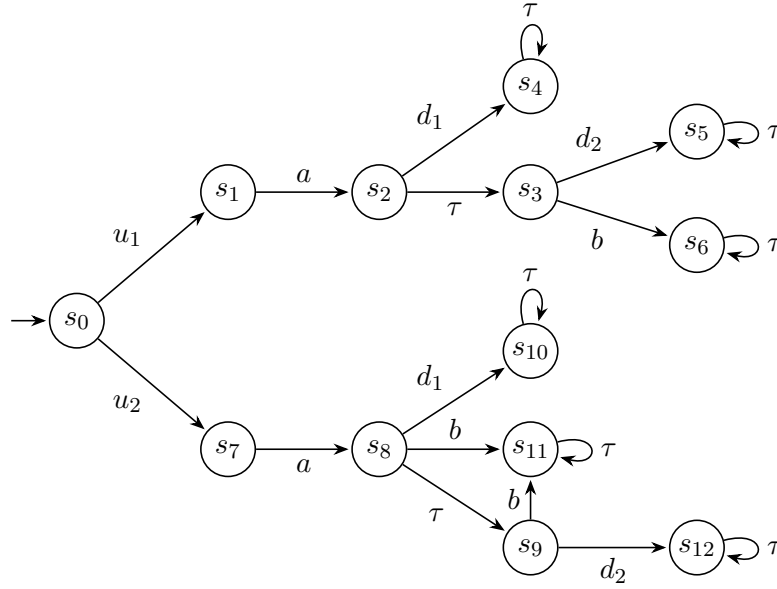


Fig. 3.4: Ejemplo 4. Adaptado de van Glabbeek y Weijland [15]. Las acciones τ son locales no controlables (inertes) en todos los casos. Los estados $s_4, s_5, s_6, s_{10}, s_{11}, s_{12}$ son trivialmente equivalentes para ambas nociones de equivalencia.

caso, vale la cláusula ya que $s_8 \xrightarrow{d_1} s_{10}$. Luego, tenemos la transición $s_2 \xrightarrow{\tau} s_3$. Como τ es no controlable, entonces podemos aplicar la cláusula 1 de WSOE y ahora debemos observar si $s_3 \sim_W s_8$. Observemos las transiciones de s_3 . Por un lado, $s_3 \xrightarrow{d_2} s_5$, como d_2 es controlable, aplicamos la cláusula 2 de WSOE y vemos que hay un camino $s_8 \xrightarrow{\tau} s_9 \xrightarrow{d_2} s_{12}$, y vale $s_5 \sim_W s_{12}$ de forma trivial, y que $\tau \in \Upsilon$ por definición. La condición que se pone para las $\tau \in \Sigma_c$ se cumple por vacuidad. Por último, observemos la última transición restante, $s_3 \xrightarrow{b} s_6$. Como existe el camino $s_8 \xrightarrow{b} s_{11}$, y $s_{11} \sim_W s_6$ de forma trivial y el camino no atraviesa ningún estado intermedio, podemos aplicar sencillamente la cláusula 2 de la definición de WSOE.

Observemos ahora las transiciones salientes de s_8 . Podemos ver que todas las acciones son alcanzables desde s_2 , directamente o a través de únicamente un estado s_3 . Es fácil en este caso ver que todas las transiciones cumplen con lo pedido en la definición de WSOE. Por lo tanto, vale que $s_2 \sim_W s_8$.

En conclusión, $s_2 \sim_W s_8$, mientras que $s_2 \not\sim_B s_8$. Este caso prueba que WSOE y Branching Bisimilarity pueden producir resultados diferentes a partir de un mismo ejemplo, por las diferencias en las nociones de equivalencia que definen. Es decir, dados los estados $x_1 \sim_W x_2$, no siempre vale que $x_1 \sim_B x_2$ y queda demostrado con el Ejemplo 4. Sin embargo, en la próxima sección demostraremos formalmente que si dos estados x_1, x_2 son Branching Equivalent, entonces debería valer que los estados x_1, x_2 son también equivalentes en la definición de WSOE.

3.4. Teorema de equivalencia y demostración

Luego de explorar la intuición detrás de la proposición, formalizamos la equivalencia entre WSOE y Branching Bisimilarity. Queremos ver que con la definición de acciones

inertes restringida a $\tau = \Upsilon_u$ entonces

$$\forall x_1, x_2 \in S | x_1 \sim_B x_2 \implies x_1 \sim_W x_2$$

Teorema 3.4.1. *Sea $M = (S, \Sigma, \rightarrow, \hat{s})$ un LTS y tomemos como conjunto de acciones inertes de la Definición 3.1.1 al conjunto $\tau = \Upsilon_u$ de acciones locales no controlables. Entonces $\sim_B \subseteq \sim_W$; es decir, para todo $x_1, x_2 \in S$, si $x_1 \sim_B x_2$ entonces $x_1 \sim_W x_2$.*

Demostración. Sea entonces R una relación Branching Bisimilar de equivalencia simétrica, y sea $(x_1, x_2) \in R$. Tomemos $x_1 \sim_B x_2$ y verifiquemos las dos cláusulas de la Definición 3.2.1 para ver si vale que $x_1 \sim_W x_2$.

Cláusula 1. Observemos el caso donde se intenta simular una transición no controlable. Sea $x_1 \xrightarrow{u} y_1$ con $u \in \Sigma_u$. Como $x_1 \sim_B x_2$, esta transición cae en alguna de las dos cláusulas de la Definición 3.1.1:

- Si vale la cláusula 1, entonces estamos observando una transición inerte. Es decir, sabemos que $u = \tau$ y $y_1 R x_2$. Por lo tanto, $u \in \tau = \Upsilon_u$. Como u es una acción local no controlable, se puede aplicar la cláusula 1 de la Definición 3.2.1, ya que u es exactamente una de las etiquetas admitidas en los tramos $\xrightarrow{\tau}$ de la definición. Así, la cláusula 1 de WSOE se satisface con el camino vacío, tomando $y_2 = x_2$, y como $y_1 R x_2$, vale $y_1 R y_2$.
- Si vale la cláusula 2, entonces existe una secuencia $x_2 \xrightarrow{\tau} \dots \xrightarrow{\tau} x'_2 \xrightarrow{u} y_2$ con $x_1 R x'_2$ y $y_1 R y_2$. Como todos los pasos intermedios son $\tau \in \Upsilon_u$, esta secuencia es un camino admisible para la cláusula 1 de WSOE y $y_1 R y_2$.

En ambos casos, la cláusula 1 de WSOE se cumple.

Cláusula 2. Observemos el caso donde se intenta simular una transición controlable. Sea $x_1 \xrightarrow{c} y_1$ con $c \in \Sigma_c$. Como $\tau = \Upsilon_u$ no contiene acciones controlables, $c \neq \tau$ y la cláusula 1 de branching no puede aplicarse. Tomemos la cláusula 2, entonces existe una secuencia $x_2 \xrightarrow{\tau} \dots \xrightarrow{\tau} x'_2 \xrightarrow{c} y_2$ con $x_1 R x'_2$ y $y_1 R y_2$. Esta secuencia satisface el camino de la cláusula 2 de WSOE, ya que los pasos intermedios cumplen $\tau_i \in \Upsilon_u \subseteq \Upsilon$. Como $\Upsilon_u \cap \Sigma_c = \emptyset$, ningún paso intermedio es controlable y por lo tanto la condición “si $\tau_i \in \Sigma_c$ entonces $x_1 R x'_2$ ” se cumple por vacuidad.

Queda demostrado que tomando $\tau = \Upsilon_u$, entonces vale $x_1 \sim_B x_2 \implies x_1 \sim_W x_2$. $\square$

4. EL ALGORITMO $O(M \log N)$ DE GROOTE ET AL.

En este capítulo se realizará un recorrido teórico por el algoritmo presentado por Groote et al. [20], dando una intuición de cómo funciona la idea general. Luego, en el Capítulo 5, se explicarán más detalles de la implementación del algoritmo.

Tal como se introdujo previamente, el algoritmo utilizado en el trabajo de Gagliardi et al. [10] para obtener, a partir de un LTS, una minimización según la equivalencia de la Definición 3.2.1 fue el presentado por Mohajerani et al. [11], que tiene una complejidad de $O(nm)$. Analicemos el algoritmo presentado por Groote et al. para entender cómo funciona y luego comprender qué optimizaciones se utilizan para alcanzar la complejidad de $O(m \log n)$.

4.1. Terminología

El algoritmo se basa principalmente en utilizar particiones de conjuntos de estados y particiones de conjuntos de transiciones. Antes, definamos qué es una partición.

Definición 4.1.1 (Partición). Para un conjunto X , una partición Π de X es un recubrimiento disjunto de X , es decir, $\Pi = \{B_i \subseteq X \mid B_i \neq \emptyset, 1 \leq i \leq k\}$ tal que $B_i \cap B_j = \emptyset$ para todo $1 \leq i < j \leq k$ y $X = \bigcup_{1 \leq i \leq k} B_i$.

El algoritmo usa dos particiones principales. La partición Π_s es una partición de los estados del LTS. Nos referimos a los elementos de Π_s como *blocks*, denotados usando la letra B . La partición Π_t particiona las transiciones no inertes del LTS. Nos referimos a los elementos de Π_t como *bunches*, denotados usando la letra T .

La partición de estados Π_s registra el conocimiento actual sobre los estados branching bisimilares: dos estados están en bloques diferentes si y solo si el algoritmo encontró una prueba de que no son branching bisimilares. La idea principal detrás del algoritmo es refinar iterativamente Π_s hasta llegar a la partición más chica cuyos bloques sean clases de estados branching bisimilares. En cada iteración, un conjunto de transiciones con etiqueta a y bloque destino B' se selecciona como el splitter, y para cada bloque B se determina si un subconjunto estricto de sus estados puede alcanzar silenciosamente una transición en el splitter. Si este es el caso, B se divide basándose en este criterio.

La partición de transiciones Π_t registra el conocimiento actual sobre los splitters. Para cada bunch de transiciones, inicialmente asumimos que sus transiciones pueden simularse entre sí de a pares, es decir, que pueden cumplir el papel de las transiciones $s \xrightarrow{a} s'$ y $t' \xrightarrow{a} t''$ de la Definición 3.1.1, hasta que se demuestre lo contrario. El algoritmo mantiene el invariante (formalizado en el Invariante 4.2.1) de que siempre que un estado en un bloque tiene una transición en un bunch, entonces cada estado en ese bloque puede alcanzar de forma inerte una transición en el mismo bunch, de tal manera que se satisfaga la condición de la Definición 3.1.1. Siempre que un bloque y un bunch particular satisfacen este invariante, decimos que el bloque es estable con respecto a ese bunch.

Sin embargo, la Definición 3.1.1 viene con algunas restricciones sobre la transición $t' \xrightarrow{a} t''$: primero, necesita tener la misma etiqueta de acción que $s \xrightarrow{a} s'$, y segundo, s' y t'' necesitan estar en el mismo bloque destino. Si se violan estas restricciones, parece que nuestra suposición inicial sobre el bunch era incorrecta, e intentamos corregir esto

dividiendo el bunch en partes que sí satisfagan las restricciones. Hacemos esto separando una action-block-slice, un subconjunto de transiciones en un bunch con la misma etiqueta de acción y el mismo bloque destino, y colocándola en un nuevo bunch. El nuevo bunch se denomina splitter primario, el resto del bunch del cual se separó se denomina splitter secundario. Después de esta división, los bloques en Π_s necesitan dividirse con respecto a ambos splitters para restablecer el invariante.

El algoritmo utiliza una serie de términos derivados de las dos particiones Π_s y Π_t . Para bunches $T \in \Pi_t$, acciones $a \in \Sigma$ y bloques $B, B' \in \Pi_s$, tenemos:

- Las *block-bunch-slices*, es decir, las transiciones en T que comienzan en B : $T_{B \rightarrow} = \{s \xrightarrow{b} s' \in T \mid s \in B\}$.
- Las *action-block-slices*, es decir, las transiciones en T que tienen etiqueta a y terminan en B' : $T_{\rightarrow B'} = \{s \xrightarrow{a} s' \in T \mid s' \in B'\}$.
- Una *block-bunch-slice* intersecada con una *action-block-slice*:

$$T_{B \rightarrow B'} = T_{B \rightarrow} \cap T_{\rightarrow B'} = \{s \xrightarrow{a} s' \in T \mid s \in B \wedge s' \in B'\}.$$
- Los *estados bottom* de un bloque, es decir, los estados sin una transición inerte saliente:

$$Bottom(B) = \{s \in B \mid \neg \exists s' \in B. s \xrightarrow{\tau} s'\}.$$
- Los estados en un bloque con una transición en un bunch: $B \xrightarrow{\tau} = \{s \mid s \xrightarrow{a} s' \in T_{B \rightarrow}\}$.
- Los bloques divisibles por una *action-block-slice*:

$$splittableBlocks(T_{\rightarrow B'}) = \{B \in \Pi_s \mid \emptyset \subset T_{B \rightarrow B'} \subset T_{B \rightarrow}\}.$$
- El número de *action-block-slices* contenidas en un bunch:

$$\#aB'(T) = |\{T_{\rightarrow B'} \mid a \in \Sigma, B' \in \Pi_s\} \setminus \{\emptyset\}|.$$
- Si $B, B' \in \Pi_s$ y $B \neq B'$, entonces $B \xrightarrow{\tau} B'$ son las transiciones τ no inertes entre B y B' .
- Las transiciones salientes de un bloque: $B_{\rightarrow} = \{s \xrightarrow{a} s' \mid s \in B, a \in \Sigma \text{ y } s' \in S\}$.
- Las transiciones entrantes de un bloque: $B_{\leftarrow} = \{s \xrightarrow{a} s' \mid s \in S, a \in \Sigma \text{ y } s' \in B\}$.

El algoritmo itera sobre Π_s y Π_t . Cuando Π_t se modifica, el invariante necesita ser restablecido dividiendo bloques. Para marcar los bloques que necesitan ser divididos, clasificamos las *block-bunch-slices* en *estables* e *inestables*. Una *block-bunch-slice* $T_{B \rightarrow}$ es estable si sabemos que no es un splitter para ningún bloque en Π_s . De lo contrario, es inestable y se debe comprobar si el bloque B debe ser dividido. Si una *block-bunch-slice* es inestable, se almacena en la splitter list, ya sea como un splitter *primario* o como un splitter *secundario*. Además, si una *block-bunch-slice* es inestable, distinguimos sus transiciones entre *marcadas* y *no marcadas*. Estas marcas se utilizan para determinar qué estados bottom de un bloque tienen una transición en un bunch particular, y son esenciales para la división eficiente de los bloques. Denotamos las transiciones marcadas de una *block-bunch-slice* $T_{B \rightarrow}$ como $Marked(T_{B \rightarrow})$.

4.2. Idea general

Como primer paso, el LTS es preprocesado para contraer las componentes τ -fuertemente conexas (τ -SCCs) en un único estado. Luego, el algoritmo refina iterativamente las particiones Π_s y Π_t . El objetivo principal es garantizar que, si un bloque B tiene una transición en una action-block-slice $T_{a,B'}$, entonces cada estado bottom de B tiene una transición en $T_{a,B'}$. Cuando esto vale para todos los bloques y todas las action-block-slices, cada bloque resulta ser exactamente una clase de estados branching bisimilares. Alcanzar ese objetivo de forma directa sería costoso, así que el algoritmo mantiene en su lugar un invariante más débil, formulado sobre los bunches en vez de sobre las action-block-slices: si un bloque B tiene una transición en un bunch T , entonces cada estado bottom de B tiene una transición en el mismo bunch T . Este invariante, combinado con la condición de que todo bunch contenga una única action-block-slice, implica el objetivo principal. Formalmente, el invariante es el siguiente:

Invariante 4.2.1. Π_s es estable bajo Π_t , es decir, si un bunch $T \in \Pi_t$ contiene una transición con su estado origen en un bloque B de Π_s , entonces cada estado bottom en el bloque B tiene una transición en el bunch T (de hecho, en el block-bunch-slice $T_{B \rightarrow}$).

Ahora bien, si un bunch contiene múltiples action-block-slices (lo denominamos un bunch no trivial), para acercarnos a nuestro objetivo principal debemos refinar Π_t separando una action-block-slice. Al mismo tiempo, para preservar el invariante principal, cuando dividimos un bunch necesitamos reflejar este cambio en los bloques. Por lo tanto, los bloques que tenían una transición alcanzable de forma inerte en el bunch original se dividen en subbloques según puedan alcanzar de forma inerte el nuevo bunch, el resto del bunch original, o ambos. Esta idea se desarrolla en el Algoritmo 1. Primero ilustramos un paso del algoritmo utilizando el ejemplo de la Figura 4.1.

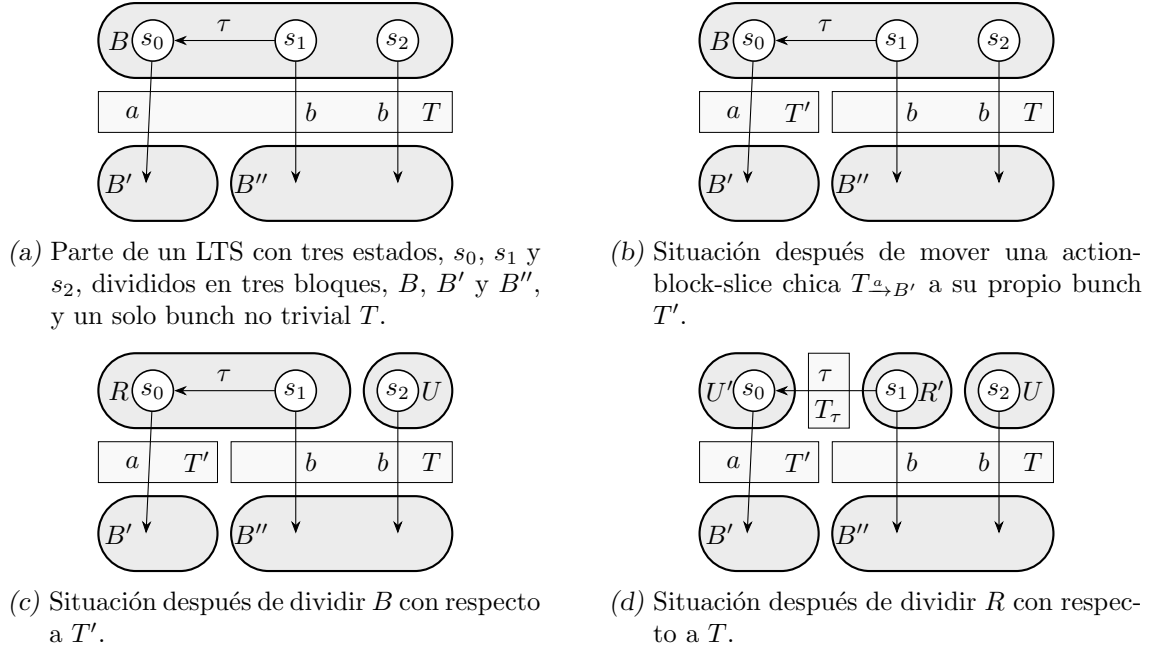


Fig. 4.1: Un paso del algoritmo de branching bisimulation.

Analicemos cómo se comporta un paso del algoritmo de branching bisimulation para una parte de un LTS, ilustrada en la Figura 4.1a. Tenemos tres estados, s_0 , s_1 y s_2 , con una transición inerte $s_1 \xrightarrow{\tau} s_0$, y varias transiciones en el bunch T . Notemos que T es no trivial: contiene dos action-block-slices, $T \xrightarrow{a}_{B'}$ y $T \xrightarrow{b}_{B''}$.

Para acercarnos a la situación en la que cada bunch contiene exactamente una action-block-slice, el algoritmo separa una action-block-slice chica de T y la coloca en su propio bunch T' . Esto se muestra en la Figura 4.1b.

Ahora, el invariante principal está roto. Para arreglarlo, necesitamos dividir el bloque B con respecto a T' y a T . Primero dividimos con respecto a T' . Esto divide el bloque B en dos bloques: R , que contiene los estados que pueden alcanzar de forma inerte una transición en T' , y U , que contiene aquellos estados desde los cuales T' es inalcanzable de forma inerte. Esto ubica a s_0 y s_1 en R , y a s_2 en U , tal como se muestra en la Figura 4.1c.

El bloque U es estable con respecto a T . El bloque R , sin embargo, aún no lo es. Por lo tanto, R debe ser dividido en dos conjuntos de estados: U' serán los estados desde los cuales una transición en T es inalcanzable, y R' los estados desde los cuales una transición en T es alcanzable. Esto se muestra en la Figura 4.1d.

Ahora, Π_s es estable con respecto a Π_t y Π_t no contiene ningún bunch no trivial, por lo que alcanzamos las particiones requeridas de branching bisimilarity para esta parte del LTS.

4.3. Funcionamiento

El pseudocódigo se detalla en el Algoritmo 1. Primero, en la Línea 1, todas las τ -SCCs se contraen en un único estado cada una. Como todos los estados de una τ -SCC son branching bisimilares, esta contracción no altera el resultado; además, el preprocesamiento garantiza que no haya τ -ciclos en el LTS y que desde cada estado no bottom se pueda alcanzar un estado bottom mediante transiciones inertes.

Luego, en las Líneas 2–4, creamos las particiones iniciales Π_s y Π_t . Sea B_{vis} el conjunto de estados desde los cuales una transición visible es alcanzable de forma inerte, y sea B_{invis} el resto de los estados, entonces $\Pi_s = \{B_{\text{vis}}, B_{\text{invis}}\} \setminus \{\emptyset\}$. Inicialmente, Π_t contiene un único bunch formado por todas las transiciones no inertes. Todos los block-bunch-slices inducidos por Π_s y Π_t son estables inicialmente, es decir, para toda block-bunch-slice $T_{B \rightarrow}$ no vacía (donde B es necesariamente B_{vis}) cada estado bottom de B tiene una transición en $T_{B \rightarrow}$.

Si hay bunches no triviales, esos bunches deben dividirse hasta que terminen siendo triviales. El ciclo externo del algoritmo (Líneas 5–30) toma un bunch no trivial T de Π_t , y de él mueve una action-block-slice $T \xrightarrow{a}_{B'}$ chica (que contiene a lo sumo la mitad de las transiciones del bunch) a un bunch propio dentro de Π_t (Línea 7). De esta manera, el bunch T queda reducido a $T \setminus T \xrightarrow{a}_{B'}$.

Entonces, ahora en Π_t tenemos un bunch nuevo más chico, cuyas transiciones tienen todas la misma etiqueta y el mismo bloque destino. Esto puede romper la estabilidad de algún bloque: antes cada estado bottom del bloque tenía alguna transición en el bunch original, pero ahora esas transiciones quedaron repartidas entre los dos bunches nuevos, y puede pasar que algunos estados bottom tengan transiciones en $T \xrightarrow{a}_{B'}$ y otros solamente en $T \setminus T \xrightarrow{a}_{B'}$. Por lo tanto, debemos recuperar la estabilidad particionando esos bloques. El conjunto $\text{splittableBlocks}(T \xrightarrow{a}_{B'})$ contiene todos los bloques que tienen transiciones en

Algoritmo 1 Pseudocódigo del algoritmo para las particiones de branching bisimulation.

```

1: Encontrar las  $\tau$ -SCCs y contraerlas a un solo estado
2:  $B_{\text{vis}} := \{s \in S \mid s \text{ puede alcanzar de forma inerte alguna } s' \xrightarrow{a} s''\};$ 
    $B_{\text{invis}} := S \setminus B_{\text{vis}}$ 
3:  $\Pi_s := \{B_{\text{vis}}, B_{\text{invis}}\} \setminus \{\emptyset\}$ 
4:  $\Pi_t := \{\{s \xrightarrow{a} s' \mid a \in \Sigma \setminus \{\tau\}, s, s' \in S\} \cup B_{\text{vis}} \xrightarrow{\tau} B_{\text{invis}}\}$ 
5: mientras exista un  $T \in \Pi_t$  con  $\#aB'(T) > 1$  hacer
6:   Tomar algún  $a \in \Sigma$  y  $B' \in \Pi_s$  tal que  $|T_{\rightarrow}^{a_{\rightarrow B'}}| \leq \frac{1}{2} |T|$ 
7:    $\Pi_t := (\Pi_t \setminus \{T\}) \cup \{T_{\rightarrow}^{a_{\rightarrow B'}}, T \setminus T_{\rightarrow}^{a_{\rightarrow B'}}\}$ 
8:   para todo  $B \in \text{splittableBlocks}(T_{\rightarrow}^{a_{\rightarrow B'}})$  hacer
9:     Agregar primero  $T_{B \rightarrow}^{a_{\rightarrow B'}}$  y después  $T_{B \rightarrow} \setminus T_{B \rightarrow}^{a_{\rightarrow B'}}$  a la splitter
       list. Etiquetar  $T_{B \rightarrow}^{a_{\rightarrow B'}}$  como primaria y  $T_{B \rightarrow} \setminus T_{B \rightarrow}^{a_{\rightarrow B'}}$  como
       secundaria
10:    Marcar todas las transiciones en  $T_{B \rightarrow}^{a_{\rightarrow B'}}$ 
11:    Por cada estado que tenga tanto transiciones salientes mar-
       cadas como una transición saliente en  $T_{B \rightarrow} \setminus T_{B \rightarrow}^{a_{\rightarrow B'}}$ , marcar
       alguna de estas últimas
12:  fin para
13:  para todo  $T'_{B \rightarrow}$  en la splitter list hacer
14:     $\langle R, U \rangle := \text{split}(B, T'_{B \rightarrow})$  //  $R \supseteq B \xrightarrow{\tau}$  puede alcanzar  $T'_{B \rightarrow}$  y
       //  $U = B \setminus R$  no puede alcanzarlo
15:    Eliminar  $T'_{B \rightarrow} = T'_{R \rightarrow}$  de la splitter list
16:     $\Pi_s := (\Pi_s \setminus \{B\}) \cup (\{R, U\} \setminus \{\emptyset\})$ 
17:    si  $T'_{B \rightarrow}$  es primario entonces
18:      Eliminar  $T_{U \rightarrow} \setminus T_{U \rightarrow}^{a_{\rightarrow B'}}$  de la splitter list
19:    fin si
20:    si  $R \xrightarrow{\tau} U \neq \emptyset$  entonces
21:      Crear un nuevo bunch que contenga exactamente  $R \xrightarrow{\tau} U$ ,
       agregar  $R \xrightarrow{\tau} U = (R \xrightarrow{\tau} U)_{R \rightarrow}$  a la splitter list, y marcar
       todas sus transiciones
22:       $\langle N, R' \rangle := \text{split}(R, R \xrightarrow{\tau} U)$  //  $N \supseteq R \xrightarrow{\tau} U$  contiene los
       // nuevos estados bottom
23:      Eliminar  $R \xrightarrow{\tau} U = (R \xrightarrow{\tau} U)_{N \rightarrow}$  de la splitter list
24:       $\Pi_s := (\Pi_s \setminus \{R\}) \cup (\{N, R'\} \setminus \{\emptyset\})$ 
25:      Agregar  $N \xrightarrow{\tau} R'$  al bunch que contiene  $R \xrightarrow{\tau} U$ 
26:      Agregar  $T_{N \rightarrow}$  a la splitter list y etiquetarla como secun-
       daria
27:      Para cada estado bottom, marcar una de sus transiciones
       salientes en cada  $T_{N \rightarrow}$ 
28:    fin si
29:  fin para
30: fin mientras

```

ambos bunches nuevos, $T_{\rightarrow}^{a_{\rightarrow B'}}$ y $T \setminus T_{\rightarrow}^{a_{\rightarrow B'}}$; estos son los bloques que hay que investigar. El primer ciclo interno (Líneas 8–12) sirve para insertar todas esas block-bunch-slices en la splitter list. Las block-bunch-slices de la forma $T_{B \rightarrow}^{a_{\rightarrow B'}}$ que están en la splitter list se etiquetan como primarias, y todas las demás block-bunch-slices de esa lista se etiquetan como secundarias.

Este ciclo también marca algunas transiciones, lo cual sirve para determinar rápidamente si un estado bottom tiene una transición en un splitter secundario $T_{B \rightarrow} \setminus T_{B \rightarrow}^{a_{\rightarrow B'}}$. Las transiciones marcadas se almacenan por separado dentro de la block-bunch-slice y por

lo tanto pueden recorrerse sin gastar tiempo en las transiciones no marcadas del bloque. Esto es esencial para obtener la complejidad temporal del algoritmo, ya que se nos permite realizar una unidad de trabajo por cada transición en $T_{\rightarrow B'}$ (el más chico de los dos bunches nuevos), y como $|Marked(T_{B \rightarrow} \setminus T_{B \rightarrow B'})| \leq |T_{B \rightarrow B'}|$, no marcamos demasiadas transiciones.

En el segundo ciclo (Líneas 13–29) se toma un splitter $T'_{B \rightarrow}$ de la splitter list por vez y su bloque origen se divide en R (la parte que puede alcanzar de forma inerte alguna transición en $T'_{B \rightarrow}$) y U (la parte que no puede alcanzar $T'_{B \rightarrow}$ de forma inerte), para restablecer la estabilidad. Formalmente, la rutina $\text{split}(B, T)$ devuelve el par $\langle R, U \rangle$ definido por:

$$\begin{aligned} R &= \{s \in B \mid s \xrightarrow{\tau} s_1 \xrightarrow{\tau} \dots \xrightarrow{\tau} s_n \xrightarrow{a} s' \text{ donde } s_1, \dots, s_n \in B, s_n \xrightarrow{a} s' \in T\}, \\ U &= B \setminus R. \end{aligned} \quad (4.1)$$

En la Sección 4.4 se detalla esta función.

Si $T'_{B \rightarrow}$ era un splitter primario de la forma $T_{B \rightarrow B'}$ agregado a la splitter list en la Línea 9, entonces sabemos que U debe ser estable respecto de $T_{U \rightarrow} \setminus T_{U \rightarrow B'}$, ya que todo estado bottom de B tiene una transición en la block-bunch-slice original $T_{B \rightarrow}$, y como los estados de U no tienen transiciones en $T_{B \rightarrow B'}$, todo estado bottom de U debe tener una transición en $T_{B \rightarrow} \setminus T_{B \rightarrow B'}$. Por lo tanto, en la Línea 18 la block-bunch-slice $T_{U \rightarrow} \setminus T_{U \rightarrow B'}$ puede eliminarse de la splitter list sin necesidad de investigarla.

Algunas transiciones invisibles pueden haber pasado a ser no inertes: precisamente, las transiciones τ que van de R a U . Las nuevas transiciones no inertes todavía no formaban parte de ningún bunch de Π_t . Por eso se forma un nuevo bunch $R \xrightarrow{\tau} U$ que las contiene. Todas las transiciones de este nuevo bunch salen del bloque R y por lo tanto R es el único bloque que puede no ser estable respecto de este bunch nuevo. Para restablecer la estabilidad dividimos R en los bloques $\langle N, R' \rangle$ (Línea 22). Observemos que puede haber transiciones $N \xrightarrow{\tau} R'$ que también pasen a ser no inertes, y las agregamos al nuevo bunch $R \xrightarrow{\tau} U$.

En N , es decir, el conjunto de estados que pueden alcanzar de forma inerte alguna transición en $R \xrightarrow{\tau} U$, algunos estados son nuevos estados bottom, mientras que R' contiene todos los estados bottom originales de R . Los bloques que contienen nuevos estados bottom pueden volverse inestables respecto de cualquier block-bunch-slice. Por lo tanto, hay que restablecer la estabilidad respecto de todas esas block-bunch-slices, y en consecuencia todas las block-bunch-slices que salen del bloque N se colocan en la splitter list. Marcamos una de las transiciones de cada nuevo estado bottom para poder encontrar los estados bottom con una transición en $T_{N \rightarrow}$ en tiempo proporcional a la cantidad de esos nuevos estados bottom.

Este ciclo se repite hasta que cada bunch coincide con una única action-block-slice. Ilustramos el algoritmo en el siguiente ejemplo. Notemos que el ejemplo también ilustra algunos de los detalles de la subrutina split , que se discute en detalle en la Sección 4.4.

Observemos la Figura 4.2a, el bloque B es estable respecto de los bunches T y T' . Separamos de T un bunch chico $T_{\rightarrow B'}$ y, como consecuencia, B necesita ser reestabilizado. Los bunches que inicialmente se colocan en la splitter list son $T_{\rightarrow B'}$ y $T \setminus T_{\rightarrow B'}$. Al colocarlos en la splitter list, todas las transiciones de $T_{B \rightarrow B'}$ quedan marcadas: las m de la Figura 4.2b. Además, para los estados que tienen transiciones tanto en $T_{\rightarrow B'}$ como en $T \setminus T_{\rightarrow B'}$, se marca una de esas transiciones del segundo bunch: las m de la Figura 4.2b.

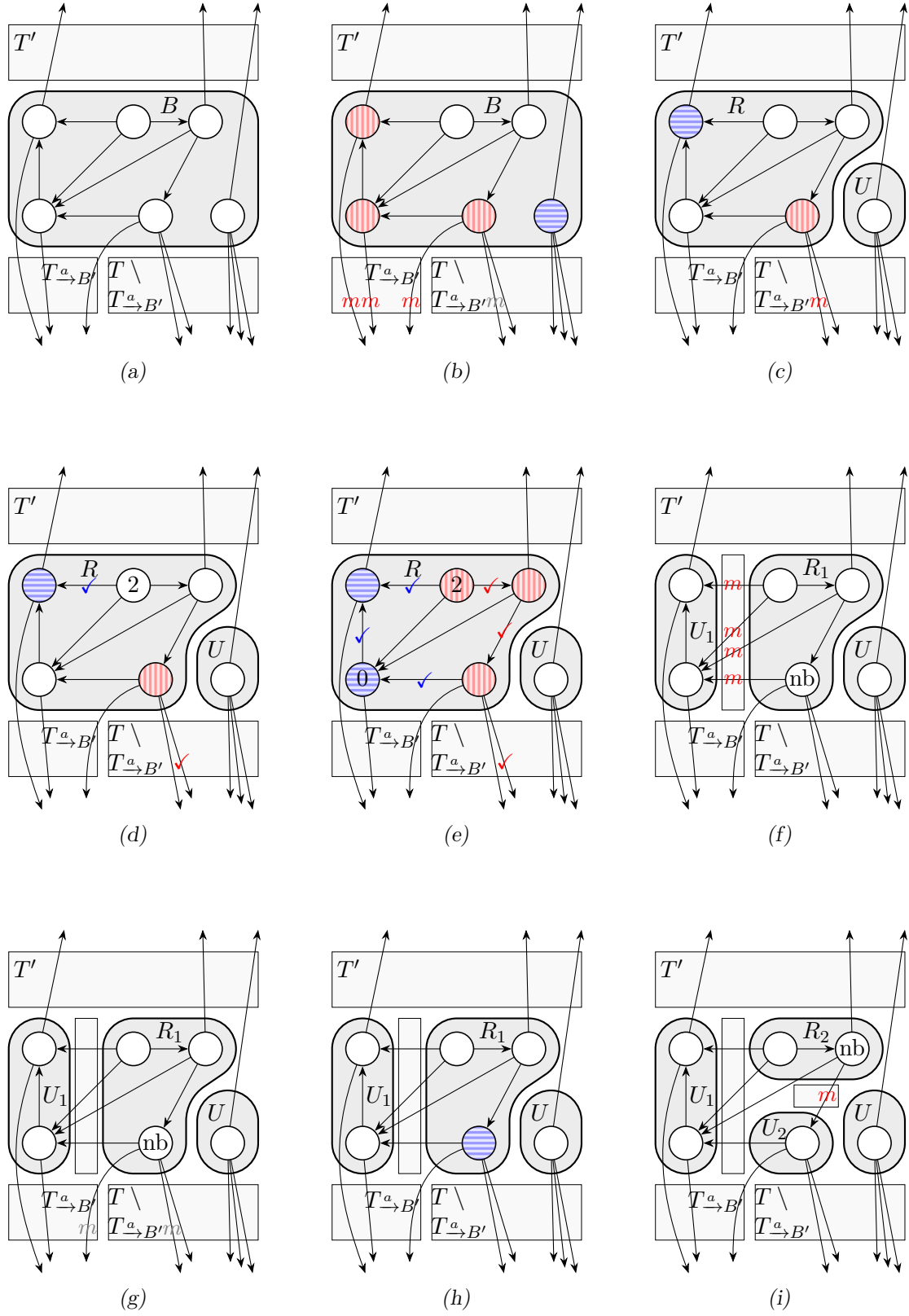


Fig. 4.2: Ilustración de la separación de una action-block-slice chica de T y de la estabilización del bloque B respecto de los nuevos bunches $T^a_{\rightarrow B'}$ y $T \setminus T^a_{\rightarrow B'}$, tal como se explica en el Ejemplo 4.3.

Primero dividimos B respecto del splitter primario $T_{\rightarrow B'}$ en R , los estados que pueden alcanzar $T_{\rightarrow B'}$ de forma inerte, y U , los estados que no pueden. En la Figura 4.2b, los estados que ya se sabe que están destinados a R se indican con $\oplus$, y los que ya se sabe que están destinados a U se indican con $\ominus$. Inicialmente, todos los estados con una transición saliente marcada están destinados a R , y el estado bottom restante de B está destinado a U . La subrutina **split** extiende los conjuntos R y U hacia atrás mediante dos corrutinas, marcando un estado como destinado a R si alguno de sus sucesores ya está en R , y marcándolo como destinado a U si todos sus sucesores están en U . En este caso, el estado de U no tiene ninguna transición inerte entrante, por lo que su corrutina termina inmediatamente y todos los demás estados pertenecen a R . El bloque B se divide en R y U . El bloque U resultante es estable respecto de $T_{\rightarrow B'}$ y de $T \setminus T_{\rightarrow B'}$. Los conjuntos R y U resultantes se muestran en la Figura 4.2c.

Todavía nos falta dividir R respecto de $T \setminus T_{\rightarrow B'}$, digamos en R_1 y U_1 . Para esto usamos las transiciones marcadas de $T \setminus T_{\rightarrow B'}$ como punto de partida para calcular todos los estados bottom que pueden alcanzar una transición en $T \setminus T_{\rightarrow B'}$. Esto garantiza que el tiempo que usamos es proporcional a $|T_{\rightarrow B'}|$. Inicialmente hay un estado destinado a R_1 , marcado con $\oplus$ en la Figura 4.2c, y un estado destinado a U_1 , marcado con $\ominus$ en la misma figura. Ahora ejecutamos ambas corrutinas de **split** simultáneamente. La Figura 4.2d muestra la situación luego de que ambas corrutinas consideraron una transición: la corrutina de U_1 (la que calcula los estados que no pueden alcanzar de forma inerte una transición en $T \setminus T_{\rightarrow B'}$) inicializó en 2 el contador *untested* de uno de los estados, en la Línea 9ℓ (es decir, en la columna izquierda) del Algoritmo 2, porque dos de sus transiciones inertes salientes todavía no fueron consideradas. La corrutina de R_1 (la que calcula los estados que sí pueden alcanzar de forma inerte una transición en $T \setminus T_{\rightarrow B'}$) revisó la transición no marcada del splitter $T_{R \rightarrow} \setminus T_{R \rightarrow B'}$. Como esta última corrutina terminó de visitar las transiciones no marcadas del splitter, la corrutina de U_1 ya no necesita ejecutar el ciclo de test lento de las Líneas 13ℓ–17ℓ del Algoritmo 2. En la Figura 4.2e se muestra la situación después de dos pasos más de las corrutinas. Cada una visitó dos transiciones adicionales. Hay dos estados nuevos destinados a R_1 , marcados con $\oplus$, y hay un estado destinado a U_1 con 0 transiciones inertes restantes, del cual sabemos inmediatamente que no tiene ninguna transición en $T \setminus T_{\rightarrow B'}$; está marcado con $\ominus$. En este punto, la corrutina de R_1 se aborta, ya que su conjunto contiene más de $\frac{1}{2} |R|$ estados, y se visitan las transiciones entrantes restantes de los estados de U_1 . Esto no extiende más a U_1 , porque todos los estados ya están destinados a R_1 o a U_1 .

El resultado de la división se muestra en la Figura 4.2f. Notemos que algunas transiciones inertes pasan a ser no inertes, por lo que se crea un nuevo bunch con las transiciones $R_1 \xrightarrow{\tau} U_1$, y todas ellas quedan marcadas con m .

A continuación tenemos que dividir R_1 respecto de este nuevo bunch, en el conjunto de estados N_1 que pueden alcanzar de forma inerte una transición del nuevo bunch, y el conjunto R'_1 que no puede alcanzarlo de forma inerte. En este caso, todos los estados de R_1 tienen una transición saliente marcada, por lo que $N_1 = R_1$ y $R'_1 = \emptyset$. La corrutina que calcula el conjunto de estados que no pueden alcanzar de forma inerte una transición del bunch termina inmediatamente, porque no hay transiciones que considerar.

Observemos que $R_1 (= N_1)$ tiene un nuevo estado bottom. Esto significa que la estabili-

dad de R_1 respecto de cualquier bunch ya no está garantizada, y hay que restablecerla. Por lo tanto, consideramos todos los bunches en los que R_1 tiene una transición saliente. Agregamos $T_{R_1 \rightarrow B'}$, $T_{R_1 \rightarrow} \setminus T_{R_1 \rightarrow B'}$ y $T'_{R_1 \rightarrow}$ a la splitter list como splitters secundarios, y marcamos con m una de las transiciones salientes de cada estado bottom en cada uno de esos bunches. Esta situación se muestra en la Figura 4.2g.

En este caso, R_1 es estable respecto de $T_{R_1 \rightarrow B'}$ y de $T_{R_1 \rightarrow} \setminus T_{R_1 \rightarrow B'}$, es decir, todos los estados de R_1 pueden alcanzar de forma inerte una transición en ambos bunches. En los dos casos esto se observa inmediatamente después de la inicialización de split, ya que el conjunto de estados que no pueden alcanzar de forma inerte una transición en esos bunches es inicialmente vacío, y la corrutina correspondiente termina de inmediato.

Consideremos entonces la división de R_1 respecto de $T'_{R_1 \rightarrow}$. Esto da lugar a R_2 , el conjunto de estados que pueden alcanzar de forma inerte una transición en T' , y U_2 , el conjunto de estados que no pueden alcanzar de forma inerte una transición en T' . Notemos que no hay transiciones marcadas en $T'_{R_1 \rightarrow}$, así que inicialmente todos los estados bottom de R_1 están destinados a U_2 (marcados con $\ominus$ en la Figura 4.2h), y no hay ningún estado destinado a R_2 . Luego empezamos a dividir R_1 . En la corrutina de R_2 , primero agregamos a R_2 los estados con una transición no marcada en $T'_{R_1 \rightarrow}$, en la Línea 4r (es decir, en la columna derecha del Algoritmo 2), y después hay que considerar todos los predecesores del nuevo estado bottom. Cuando split termina, no hay estados adicionales en U_2 , y los estados restantes terminan en R_2 .

La situación después de dividir R_1 en R_2 y U_2 se muestra en la Figura 4.2i. Una de las transiciones inertes pasa a ser no inerte, y está marcada con m . Además, R_2 contiene un nuevo estado bottom: es el estado de R_2 con una transición hacia T' , y está marcado con nb.

Nos queda seguir estabilizando R_2 respecto del bunch $R_2 \xrightarrow{\tau} U_2$, lo cual no genera una nueva división, y reestabilizar R_2 respecto de todos los bunches en los que tiene una transición saliente. Esto tampoco genera nuevas divisiones, por lo que la situación de la Figura 4.2i, luego de quitar las marcas de las transiciones en $R_2 \xrightarrow{\tau} U_2$, es el resultado final de la división.

4.4. La función split

La función $\text{split}(B, T)$, presentada en el Algoritmo 2, refina el bloque B en dos subbloques, R y U , donde R contiene aquellos estados de B que pueden alcanzar de forma inerte una transición en T , y U contiene los estados que no pueden, tal como se especifica formalmente en la Ecuación (4.1).

Estos dos conjuntos se calculan mediante dos corrutinas que se ejecutan en lockstep: las dos corrutinas comienzan la misma cantidad de iteraciones de sus ciclos, de modo que el overhead es a lo sumo proporcional al tiempo de la más rápida de las dos, y todo el trabajo hecho en ambas corrutinas puede atribuirse al más chico de los dos subbloques R y U .

Como precondition, la función requiere que el marcado de las transiciones en $T_{B \rightarrow}$ sea tal que los estados bottom de B que tienen una transición saliente en $T_{B \rightarrow}$ también tengan una transición saliente marcada en $T_{B \rightarrow}$. Formalmente, usando la notación $X \xrightarrow{T}$ para el conjunto de estados de X con una transición saliente en T ,

$$\text{Bottom}(B) \xrightarrow{\text{Marked}(T_{B \rightarrow})} = \text{Bottom}(B) \xrightarrow{T_{B \rightarrow}}.$$

Los conjuntos iniciales se calculan de la siguiente manera. Inicialmente, todos los estados de $B \xrightarrow{\text{Marked}(T)}$, es decir, todos los estados que son origen de una transición marcada en T , se colocan en R . Todos los estados bottom que inicialmente no están en R se colocan en U . Observemos que estos conjuntos iniciales pueden calcularse en tiempo $O(|\text{Marked}(T)|)$ agrupando los estados bottom que tienen transiciones marcadas.

Los conjuntos se extienden en las corrutinas de la siguiente manera. Para los estados de R , primero se agregan los estados de $B \xrightarrow{T \setminus \text{Marked}(T)}$ que todavía no estaban en R . Estos son todos los orígenes de una transición no marcada en T . Usando *backwards inert reachability*, R se extiende hasta que no se pueden agregar más estados o bien R contiene más de la mitad de los estados de B .

Para identificar los estados de U , observemos que un estado t está en U si todos sus sucesores inertes están en U y no tiene ninguna transición en $T_{B \rightarrow}$. La primera condición se satisface trivialmente para los estados bottom. Para calcular U , usamos un contador $\text{untested}[t]$ para cada estado no bottom t , que registra la cantidad de transiciones inertes salientes hacia estados de los que todavía no se sabe si están en U . Si $\text{untested}[t] = 0$, esto significa que todos los sucesores inertes de t están garantizadamente en U , así que, siempre que t no tenga una transición en $T_{B \rightarrow}$, t también se agrega a U . Para cubrir la posibilidad de que todas las transiciones inertes de t hayan sido visitadas antes de que todos los estados que son origen de una transición en $T_{B \rightarrow}$ se hayan agregado a R , revisamos todas las transiciones no inertes de t para determinar si no están en $T_{B \rightarrow}$, en las Líneas 13ℓ–17ℓ, es decir, en la columna izquierda del Algoritmo 2.

La corrutina que termina primero, siempre que su cantidad de estados no exceda $\frac{1}{2}|B|$, calculó completamente el subbloque más chico resultante del refinamiento, y la otra corrutina puede abortarse.

En detalle, la complejidad temporal de $\langle R, U \rangle := \text{split}(B, T)$ es:

- $O(|R_{\rightarrow}| + |R_{\leftarrow}|)$, si $|R| \leq |U|$, y
- $O(|\text{Marked}(T)| + |U_{\rightarrow}| + |U_{\leftarrow}| + |(Bottom(R) \setminus Bottom(B))_{\rightarrow}|)$, si $|U| \leq |R|$.

Esta complejidad puede deducirse de la siguiente manera. Primero, notemos que ejecutamos las corrutinas en lockstep. Eso significa que ejecutarlas implica un overhead que es a lo sumo proporcional al tiempo de la más rápida de las dos. Tan pronto como una de las corrutinas encuentra más de $\frac{1}{2}|B|$ estados, se sabe que su subbloque es demasiado grande, por lo que esa corrutina se aborta. Esto solo reduce el overhead. Por lo tanto, basta con mostrar que se satisface la cota de tiempo del subbloque más chico. Notemos que si ambos subbloques tienen el mismo tamaño, $|R| = |U| = \frac{1}{2}|B|$, la más rápida de las dos termina primero, así que ambas cotas de tiempo valen.

Consideremos el caso $|R| \leq |U|$. Observemos que $|\text{Marked}(T)| \leq |R_{\rightarrow}|$, y que todo el trabajo se atribuye a la corrutina de la derecha del Algoritmo 2, por lo que obtenemos $O(|R_{\rightarrow}| + |R_{\leftarrow}|)$ directamente de la corrutina de R .

Consideremos ahora el caso $|U| \leq |R|$. Entonces usamos tiempo $O(|\text{Marked}(T)|)$ para la Línea 2, y tiempo $O(|U_{\leftarrow}|)$ para todo lo demás, excepto las Líneas 13ℓ–17ℓ. Para estas últimas líneas distinguimos dos casos. Si resulta que t no tiene ninguna transición $t \xrightarrow{\alpha} u \in T$, entonces es un estado de U , así que podemos atribuir el tiempo a $O(|U_{\rightarrow}|)$. En caso contrario, es decir, si existe alguna $t \xrightarrow{\alpha} u \in T$, es un estado de R . ¿Cómo se contabiliza un estado de R en la corrutina que se supone que encuentra U ? La solución es que t es un nuevo estado bottom. Tenía algunas transiciones inertes en B , pero ahora todas están en $R \xrightarrow{T} U$. Así que atribuimos el tiempo a las transiciones salientes de los nuevos

Algoritmo 2 Refinamiento de un bloque respecto de un splitter.

1: función <code>split</code> (bloque B , block-bunch-slice T)		
2: $R := B \xrightarrow{\text{Marked}(T)}$; $U := \text{Bottom}(B) \setminus R$		$\} O(\text{Marked}(T))$
3: comienzo de corrutinas		
4: Definir $\text{untested}[t]$ indefinido para todo $t \in B$	$R := R \cup B \xrightarrow{T \setminus \text{Marked}(T)}$	
5: para todo $s \in U$ mientras $ U \leq \frac{1}{2} B $ hacer	para todo $s \in R$	$\} O(1) \text{ o } O(R_{\rightarrow})$
6: para todo $t \xrightarrow{\tau} s$ inerte hacer	mientras $ R \leq \frac{1}{2} B $ hacer	
7: si $t \in R$ entonces saltar estado t	para todo $t \xrightarrow{\tau} s$ inerte hacer	
8: si $\text{untested}[t]$ indefinido entonces		$\} O(U_{\leftarrow}) \text{ o }$
9: $\text{untested}[t] := \{t \xrightarrow{\tau} u \mid u \in B\} $		$O(R_{\leftarrow})$
10: fin si		
11: $\text{untested}[t] := \text{untested}[t] - 1$		
12: si $\text{untested}[t] > 0$ entonces saltar t		
13: si $B \xrightarrow{\tau} \not\subseteq R$ entonces		
14: para todo $t \xrightarrow{\alpha} u$ no inerte hacer		$\} O(U_{\rightarrow} +$
15: si $t \xrightarrow{\alpha} u \in T$ entonces saltar t		$ (Bottom(R) \setminus$
16: fin para		$Bottom(B))_{\rightarrow})$
17: fin si		
18: Agregar t a U	Agregar t a R	
19: fin para	fin para	$\} O(U_{\leftarrow}) \text{ o }$
20: fin para	fin para	$O(R_{\leftarrow})$
21: si $ U > \frac{1}{2} B $ entonces	si $ R > \frac{1}{2} B $ entonces	
22: Abortar esta corrutina	Abortar esta corrutina	
23: fin si	fin si	$\} O(1)$
24: Abortar la otra corrutina	Abortar la otra corrutina	
25: retornar $(B \setminus U, U)$	retornar $(R, B \setminus R)$	
26: fin de corrutinas		

estados bottom: $O(|(Bottom(R) \setminus Bottom(B))_{\rightarrow}|)$. Notemos que eventualmente llegamos a la situación en la que todos los estados de $B \xrightarrow{\tau}$ están en R por la Línea 4r, y esta verificación costosa se saltea.

También puede pasar que U sea vacío. En ese caso se gasta tiempo $O(|\text{Marked}(T)|)$ en `split`. La inicialización toma tiempo $O(|\text{Marked}(T)|)$ y, como U queda vacío después de la inicialización, la corrutina izquierda termina inmediatamente y la corrutina derecha se aborta. Esto último toma tiempo $O(1)$.

4.5. Detalle de la complejidad

Una vez descriptos el algoritmo principal (Algoritmo 1) y la subrutina `split` (Algoritmo 2), podemos analizar de dónde sale la cota $O(m \log n)$. Para simplificar la notación asumimos, al igual que los autores del trabajo original, que $n \leq m + 1$.

La inicialización (Líneas 1–5) se hace en tiempo $O(m)$: la contracción de las τ -SCCs es un recorrido sobre el subgrafo de transiciones τ , y para el cálculo de los conjuntos de estados iniciales se usa el supuesto $n \leq m + 1$. El costo del ciclo principal lo podemos dividir en tres:

1. la división de bunches, junto con el agregado de block-bunch-slices a la splitter list y el marcado de transiciones, que se atribuye a las transiciones del nuevo bunch chico;
2. la división de bloques, es decir, la función `split`;
3. los cálculos que hacen falta cuando aparecen nuevos estados bottom.

División de bunches. Acá se aplica el principio de procesar la mitad más chica a los bunches nuevos, y lo que hay que acotar es cuántas veces una misma transición puede formar parte de un bunch nuevo en la Línea 9.

La primera vez que eso ocurre, el bunch nuevo es una action-block-slice $T_{\rightarrow B'}$: todas sus transiciones tienen la misma etiqueta a y el mismo bloque destino B' , y como hay a lo sumo n^2 pares de estados, ese bunch contiene a lo sumo n^2 transiciones. Cada vez siguiente que se separa un bunch nuevo que contiene esa misma transición, ese bunch tiene a lo sumo la mitad del tamaño del anterior, porque la Línea 6 elige siempre una $T_{\rightarrow B'}$ con $|T_{\rightarrow B'}| \leq \frac{1}{2} |T|$. Un bunch con una sola transición ya no se puede volver a partir, así que la cantidad de veces que una transición aparece en un bunch nuevo está acotada por la cantidad de veces que se puede partir n^2 al medio hasta llegar a 1, que son $2 \log_2 n$ veces. En otras palabras, cada transición participa de un bunch nuevo $O(\log n)$ veces.

Cada una de esas veces la transición se procesa en tiempo constante, que es el $O(|T_{\rightarrow B'}|)$ de las Líneas 6–13. Sumando sobre las m transiciones, el trabajo atribuido a la división de bunches es $O(m \log n)$. Notemos que acá está la diferencia esencial con un algoritmo que recorriera todos los bunches en cada iteración: el factor $\log n$ aparece justamente porque cada transición solo puede caer en un bunch nuevo esa cantidad de veces.

Dentro de esta misma parte queda contabilizado el procesamiento de las block-bunch-slices de la forma $T_{B \rightarrow B'}$ y $T_{B \rightarrow} \setminus T_{B \rightarrow B'}$ que se agregan a la splitter list en la Línea 9: cuesta $O(|\text{Marked}(T'_{B \rightarrow})|)$ en las Líneas 14–21, y eso es a lo sumo la cantidad de transiciones en $T_{B \rightarrow B'}$, por lo que ya está cubierto.

División de bloques. Como se vio en la Sección 4.4, el costo de split depende del más chico de los dos subbloques resultantes, así que acá aplicamos procesar la mitad más chica a los bloques. El razonamiento es el mismo que antes: la primera vez que un estado cae en el subbloque más chico, ese subbloque tiene a lo sumo $n/2$ estados, y cada vez siguiente tiene a lo sumo la mitad del tamaño del anterior; como un subbloque de un solo estado ya no se parte, un estado puede estar en el subbloque más chico a lo sumo $\log_2 n$ veces. Cada vez que un estado está en el subbloque más chico podemos atribuirle tiempo proporcional a la cantidad de sus transiciones entrantes y salientes, que es exactamente lo que dan las cotas de split: $O(|R_{\rightarrow}| + |R_{\leftarrow}|)$ si $|R| \leq |U|$, y $O(|\text{Marked}(T)| + |U_{\rightarrow}| + |U_{\leftarrow}| + |(Bottom(R) \setminus Bottom(B))_{\rightarrow}|)$ si $|U| \leq |R|$.

En el Algoritmo 1 esto aparece como

$$\begin{aligned} O(|U_{\rightarrow}| + |U_{\leftarrow}|) &\text{ o } O(|R_{\rightarrow}| + |R_{\leftarrow}|) && \text{(Líneas 14–21),} \\ O(|R'_{\rightarrow}| + |R'_{\leftarrow}|) &\text{ o } O(|N_{\rightarrow}| + |N_{\leftarrow}|) && \text{(Líneas 22–25).} \end{aligned}$$

Además, como $R \xrightarrow{\tau} U \subseteq U_{\leftarrow} \cap R_{\rightarrow}$, el término $O(|R \xrightarrow{\tau} U|)$ de las Líneas 22–25 se atribuye a U o a R , al que sea más chico. Sumando sobre todos los estados: cada estado s aporta a lo sumo $O(\log n)$ veces un costo proporcional a su cantidad de transiciones entrantes y salientes, y como la suma de esas cantidades sobre todos los estados es $O(m)$, el total atribuido a la división de bloques es también $O(m \log n)$.

Nuevos estados bottom. Cuando en un bloque aparecen nuevos estados bottom, ese bloque puede volverse inestable respecto de algún bunch, así que hay que volver a poner en la splitter list todas las block-bunch-slices que salen de él y marcar transiciones en cada una (Líneas 26–27).

$Bottom(N)_{\rightarrow}$ es el conjunto de las transiciones salientes de los estados bottom de N . Cada vez que un estado s se convierte en un nuevo estado bottom, la complejidad es

proporcional a la cantidad de transiciones que salen de s , sumada sobre los nuevos estados bottom de N , tal como se ve en la anotación $O(|Bottom(N)_{\rightarrow}|)$ de las Líneas 14–27.

Un estado se puede convertir en estado bottom solo una vez a lo largo del algoritmo. Los bloques solo se refinan y las transiciones inertes solo pueden dejar de ser inertes, nunca al revés, así que una vez que un estado se queda sin transiciones inertes salientes es bottom para siempre. Por lo tanto, el trabajo total de esta parte está acotado por la suma, sobre todos los estados, de sus transiciones salientes:

$$\sum_{s \in S} |s_{\rightarrow}| = m,$$

es decir, $O(m)$.

La Línea 26 vuelve a marcar como inestables todas las block-bunch-slices que salen de N , así que su costo es proporcional a la cantidad de esas slices. Acá no alcanza con contar los nuevos estados bottom que N tiene en este momento, porque puede haber block-bunch-slices cuyas transiciones salgan únicamente de estados no bottom. Cuando N se divida respecto de una de esas slices, al menos uno de esos estados no bottom va a pasar a ser bottom, pero todavía no podemos decir cuál. Por eso el costo se contabiliza como $O(|Bottom^*(N)_{\rightarrow}|)$, donde $Bottom^*(N)$ denota los nuevos estados bottom de N presentes y futuros, es decir, contamos también a los estados que todavía no son bottom pero de los que ya sabemos que lo van a ser.

Total.

inicialización (Líneas 1–5)	$O(m)$
división de bunches	$O(m \log n)$
división de bloques	$O(m \log n)$
nuevos estados bottom	$O(m)$
total	$O(m \log n)$

El total es $O(m \log n)$, que coincide con la cota prometida por los autores. Comparada con la complejidad $O(nm)$ del algoritmo de Mohajerani et al. [11] usado hasta ahora en la síntesis composicional, la mejora en las cotas es de un factor $n/\log n$. En el Capítulo 5 se describe la implementación de este algoritmo dentro de MTSA, y qué estructuras de datos hacen falta para que cada línea del pseudocódigo respete la complejidad con la que fue descrita en esta sección.

5. IMPLEMENTACIÓN DEL ALGORITMO

5.1. Introducción

La herramienta MTSA [22] es el *framework* de desarrollo realizado en Java para varias iniciativas de LaFHIS en el contexto de la investigación sobre modelado y verificación de sistemas concurrentes, así como síntesis de controladores. Es por eso que tanto el algoritmo composicional presentado por Gagliardi et al. [10] como la versión de WSOE [11] utilizada para la composicionalidad fueron implementados en esta herramienta. Era entonces natural realizar el desarrollo del algoritmo de *branching bisimilarity* sobre el mismo *framework*. A su vez, nos permitirá comparar de forma más verídica ambos algoritmos, realizando pruebas de *benchmarks* tanto para tiempos de ejecución como para el uso de memoria. En primer lugar, se realizó la implementación del algoritmo de *branching bisimilarity* en el archivo `BranchingEquivalence.java`, creado en `mtstools/DCS/Compositional/` ya que bajo este módulo están las herramientas utilizadas por el algoritmo composicional.

A su vez, en `CompositionalApproach.java` se reemplazó la llamada original al algoritmo de WSOE por la llamada a la nueva función que utiliza el algoritmo de Groote et al. [20], reemplazando también los preprocesamientos necesarios para los algoritmos que se hacen en algunos casos fuera de las funciones principales.

La llamada al algoritmo de *branching bisimilarity* queda entonces de esta forma

```
Pair<Pi_s, Pi_t> partition
    = BranchingEquivalence.getPartitions(lts, alphabet,
    ↪ controllables, fluentGoals);

MTS<Long, String> minimizationResult =
    ↪ BranchingEquivalence.buildMinimisedMTSFromPartition(lts, tauLabels,
    ↪ controllables, partition);
```

donde en la función `getPartitions(...)` se implementa el algoritmo de *branching bisimilarity* de Groote et al. [20], presentado en el Capítulo 4, y se devuelve la partición final de estados y transiciones, y en `buildMinimisedMTSFromPartition(...)` se construye el MTS minimizado a partir de la partición calculada.

A su vez, el algoritmo recibe como parámetros el LTS en realidad a minimizar, el alfabeto local, el traductor total y los fluents objetivo. **HG** *<hay un par de estas cosas que habría que chequear si explicar antes que son o sino tenes que explicarlas acá, por ej. traductor total, fluents objetivo, etc.>*

En primer lugar, la modalidad de desarrollo fue implementar la lógica del algoritmo lo más fielmente al paper posible, es decir, sin tomar en cuenta las optimizaciones ni las cotas de complejidad analizadas en la Sección 4.5. En la primera versión (v_1), el algoritmo replica la estructura del pseudocódigo del Capítulo 4 pero reemplazando las estructuras de datos propuestas por estructuras nativas de MTSA.

5.2. Primera implementación del algoritmo

Como fue explicado en el Capítulo 4, el algoritmo de Groote et al. [20] mantiene dos estructuras de refinamiento: una partición de estados Π_s y una partición de transiciones Π_t . En Π_s se encuentran los *bloques* de estados estables tal que si dos estados están en

distintos bloques entonces **no** son *branching bisimilar*. En Π_t se encuentran los *bunches* de transiciones no-inertes, es decir, las transiciones que pueden distinguir estados. Toda la terminología que se usa en este capítulo (bloques, bunches, block-bunch-slices, action-block-slices, estados bottom, splitters primarios y secundarios, y transiciones marcadas) está definida en la Sección 4.1, y la propiedad que el refinamiento preserva es el Invariante 4.2.1. Ambas estructuras se refinan a la par: cada vez que un bloque de estados se parte en dos, se actualizan los bunches para reflejar el nuevo bloque, y cada vez que un bunch se parte en slices por (acción, bloque destino), se generan nuevos splitters para estabilizar los bloques afectados. El proceso continúa hasta llegar al punto fijo donde no quedan ni splitters pendientes ni bunches por refinar: las clases de equivalencia son entonces los bloques finales de Π_s .

La idea para esta primera implementación fue mantener la estructura del paper, es decir la del Algoritmo 1, donde el *loop* principal (Línea 5) itera sobre cada conjunto de transiciones no triviales que están en Π_t , tomando un subconjunto chico de transiciones (Línea 6). Este subconjunto de transiciones, es separado de su bunch original y añadido a Π_t como un nuevo bunch. Luego, se deben refinar los bloques de Π_s para que queden estables respecto al nuevo conjunto de transiciones obtenido.

Una decisión importante para esta primera versión fue la elección de las estructuras de datos a utilizar, ya que inciden en la complejidad final de la implementación. En esta primera versión, buscando en primer lugar comprender la lógica en sí del algoritmo y poder verificar su correctitud, se eligieron estructuras de datos simples y nativas, lo más similar posible a las del paper.

Π_s se implementó como `List<Set<Long>>`, es decir una lista de conjuntos de estados. Cada `Set<Long>` representa un bloque de estados. Π_t se implementó como `List<Set<Triple<Long, String, Long>>>`, es decir una lista de conjuntos de transiciones. Cada `Set<Triple<Long, String, Long>>` representa un bunch de transiciones no-inertes. **HG** *«Está muy bien todo esto, creo que quizá si modificas arriba los tipos de datos para hacerlos más pseudocódigo puedes hacerlo acá también»*

A su vez, se debió utilizar una estructura `Pi_t cola` para poder iterar sobre los bunches a la vez que se actualizaba la lista de pendientes a refinar. Tal como se menciona en el paper de Groote et al., se utiliza también la estructura del `splitterList` para almacenar los splitters generados durante el proceso de refinamiento, que es la *splitter list* de la Sección 4.1. A su vez, se creó la clase `Splitter` para representar cada splitter, que contiene el bloque que se va a partir, el conjunto de transiciones que cruzan bloques y un flag de *primary/secondary*, un conjunto de transiciones marcadas para identificar que esos bloques son estables y un `groupId` para identificar qué Splitters se originaron a partir de la misma división inicial.

En primer lugar, el primer preprocesamiento requerido para este algoritmo es eliminar las componentes tau fuertemente conexas, tal como indica la Línea 1 del Algoritmo 1. La implementación de este paso usa la lógica del algoritmo de Kosaraju [21], que tiene una complejidad $O(n + m)$, por lo que entra dentro del presupuesto $O(m)$ que la Sección 4.5 asigna a la inicialización. Luego, se deben inicializar las particiones, tal como indican las Líneas 2–4 del Algoritmo 1. Π_s arranca con dos bloques (B_{vis} y B_{invis}), donde B_{vis} contiene el conjunto de estados desde los cuales se puede alcanzar una transición visible y B_{invis} es su complemento. Para realizar esta inicialización, se ejecuta la función `computeBvis` que genera un grafo de predecesores, almacenando para cada transición, cuál fue su origen. Luego, identifica cuáles son visibles y realiza un DFS para propagar las acciones visibles

e identificar desde qué estados son alcanzables. Por lo tanto, la complejidad resultante de la función es también $O(n + m)$. Finalmente, B_{vis} se inicializa con el conjunto de estados identificados por esta función como visibles y B_{invis} se inicializa con el resto. Π_t es inicializado con un único bunch que contiene todas las transiciones no-inertes.

Una de las primeras estructuras que también se deben inicializar en el preprocesamiento es el `stateToBlockMap`, definido como un mapa `Map<Long, Set<Long>>`, que almacena para cada estado a qué bloque de Π_s pertenece. Esta estructura es indispensable para poder realizar el seguimiento y las modificaciones rápidas que identifican a cada estado con su bloque, ya que la lógica principal del algoritmo se basa en modificar estos bloques continuamente. A su vez, esta estructura es acompañada por la función `updateStateToBlockMap(Map<Long, Set<Long>> map, Set<Long> newBlock1, Set<Long> newBlock2)` que recibe dos bloques nuevos y actualiza el mapa para que cada estado de cada bloque nuevo quede asociado a su bloque correspondiente.

Luego del preprocesamiento y la inicialización de las estructuras de datos, el código tiene un *loop* principal que itera sobre los bunches. En cada ciclo, se toma un bunch T . Si T es un bunch trivial, es decir que todas sus transiciones con la misma acción a van a un mismo bloque, entonces el bunch T no debe ser refinado. Si T no es trivial, entonces se buscan todos los "slices" de T , es decir, se buscan y se separan los subconjuntos de T que tienen la misma acción a y bloque de destino B' . Luego, se itera sobre todos los slices de T y se separa el primer slice encontrado que cumpla que es "chico", se lo identifica almacenándolo en la estructura `chosenTransitions`. Ahora, el objetivo es dividir a T , separando las `chosenTransitions` de T y dejando el resto de T en otra estructura que llamada `newBunch`. Luego, se agrega tanto el `chosenTransitions` como el `newBunch` a Π_t y a la cola de bunches por refinar, tal como indica la Línea 7 del Algoritmo 1. Ahora, se tienen dos bunches en Π_t que no son estables respecto a ambos bunches, por lo que se deben refinar los bloques de Π_s para que lo sean. Para esto, se utiliza la función `findSplittableBlocks`, que implementa el conjunto $splittableBlocks(T \xrightarrow{a} B')$ definido en la Sección 4.1, buscando todos los bloques que contienen transiciones en los dos nuevos bunches. Por cada bloque B encontrado, se crea un splitter primario y otro secundario y se guardan en una lista `splitterList`, tal como indica la Línea 9 del Algoritmo 1. El splitter primario contiene todas las transiciones de `chosenTransitions` que salen de B , y el splitter secundario contiene todas las transiciones de `newBunch` que salen de B . A su vez, se debe marcar como transiciones necesarias a reestabilizar todas las transiciones de `chosenTransitions`, es decir, las marcas del splitter primario (Línea 10). En esta primera versión, se utiliza un mapa `Map<Splitter, Set<Triple<Long, String, Long>>>` `markings` para almacenar estas marcas, asociando a cada splitter el conjunto de transiciones que fueron marcadas para la reestabilización. Por cada estado que es origen de una transición primaria, se debe identificar si ese estado también tiene una transición secundaria, y marcar a una transición secundaria por cada estado, añadiéndola a `markings`, asociadas al splitter secundario (Línea 11).

Luego, se itera sobre la `splitterList`, tal como indican las Líneas 13–29 del Algoritmo 1. Por cada bloque B que se está observando para realizar la partición, se define el conjunto de estados R que son los estados de B que pueden alcanzar inerteemente una transición del splitter y el conjunto de estados U que son los que no. Esta división en nuevos bloques R y U se hace a través de una función "split" que se detallará más adelante. Siguiendo la lógica del paper, si el splitter actual era primario, entonces luego del split, el bloque U es estable respecto al bunch `chosenTransitions`, por lo que se debe

crear un nuevo splitter secundario para el bloque B , que solo contiene las transiciones de **chosenTransitions** que salen de R . Es decir, se reemplaza el splitter secundario de B por el nuevo splitter secundario para R , tal como indica la Línea 18. Al realizar el split de R y U , las transiciones inertes salientes de R hacia U , antes eran inertes pues estaban dentro del bloque U , pero ahora es necesario identificarlas como no-inertes, ya que ahora cruzan dos bloques separados. Para realizar esta identificación, se utiliza la función **findNewNonInertTransitions**. Estas nuevas transiciones no-inertes, conforman un nuevo bunch que se debe agregar a Π_t (Líneas 20–21). A su vez, tal como se menciona en el paper, R puede haberse vuelto inestable respecto al nuevo bunch creado. Por ello, se vuelve a utilizar la función **split** para volver a dividir R en dos bloques, N y R' . Si N y R' no son vacíos, entonces se reemplaza a R de Π_s por N y R' , tal como indican las Líneas 22–24.

Acorde con el análisis hecho en el paper, la estabilidad no se preserva cuando aparecen nuevos estados bottom, por lo que se debe estabilizar al nuevo bloque N con respecto a todos los bunches a donde N tenga transiciones salientes, tal como indican las Líneas 26–27 del Algoritmo 1. El párrafo “Nuevos estados bottom” de la Sección 4.5 explica por qué este trabajo, a pesar de recorrer todos los bunches salientes, queda acotado por $O(m)$ en total. Para ello, se identifican todos los bottom states de N y se itera sobre Π_t para encontrar todos los bunches que contienen alguna transición saliente de N . Por cada bunch encontrado, se crea un splitter secundario. Por último, se itera sobre los estados bottom identificados de N y se marca a una transición por cada estado. Estas marcas se realizan agregando a la primera transición del estado en una lista de estados **marksForRestabilization**.

Retomando, la función **split** es el corazón del algoritmo. La función **split** es la que se encarga de dividir al bloque B en los bloques R y U según la propiedad de la Ecuación (4.1), y la implementación de esta función es la que hace la diferencia en el paper para poder alcanzar la cota de la complejidad. En el paper se detalla a la función **split** como dos corutinas que se ejecutan en paralelo, tal como se describe en la Sección 4.4 y se formaliza en el Algoritmo 2. Sin embargo, con vistas a nuestro objetivo de poder realizar primero una primera implementación del algoritmo y validar su correctitud, en esta primera versión no se implementó la función como dos corutinas. La función **split** tiene la siguiente firma:

```
private static Pair<Set<Long>, Set<Long>> split(
    Set<Long> B,
    Set<Triple<Long, String, Long>> transitions,
    Set<Triple<Long, String, Long>> currentMarks,
    MTS<Long, String> toMinimise,
    Set<String> tauLabels,
    Map<Long, Set<Long>> stateToSCCMap)
```

Es decir, la función **split** recibe como argumentos el bloque B a dividir, las transiciones del splitter asociado, las marcas del splitter asociado, el LTS a minimizar, el conjunto de etiquetas identificadas como tau y el mapa de estados a bloques. A su vez, **split** devuelve un par de conjuntos de estados, el bloque R y el bloque U . En esta versión, **split** se limita a calcular el conjunto R usando *backward reachability* para las transiciones inertes, es decir, implementa únicamente la corrutina de la columna derecha del Algoritmo 2. Al no ejecutarse la corrutina de U en *lockstep*, se pierde la posibilidad de cortar el cálculo al llegar a la mitad del bloque, que es justamente de donde sale la cota de la Sección 4.5; esta limitación se retoma en la Sección 5.5.3. Para comenzar este procesamiento, todos los estados con alguna transición marcada (es decir, incluida en **currentMarks**), son puestos en el nuevo bloque R . A su vez, se recorren todas las transiciones del splitter para incluir también en R a los estados desde los cuales estas transiciones salen. Luego, se construye el

mapa `inertPredecessorsInB`, que almacena para cada estado cuáles son sus predecesores inertes. Luego, se propagan las transiciones inertes hacia atrás, utilizando una estructura `worklistR` que comienza con todos los estados de R , y por cada uno de ellos busca qué estados son predecesores inertes y los agrega a R y a la `worklist` para procesarlos también. El proceso continúa hasta que no queden predecesores inertes y por lo tanto, R sea estable. Luego, se devuelve el nuevo bloque R y el nuevo bloque U , definido como el complemento de R en B .

5.3. Primeros tests

Como se mencionó anteriormente, la estrategia fue desarrollar una primera versión fiel al algoritmo para entender la lógica y poder validar su correctitud. Por eso, luego de realizar esta primera implementación, se desarrollaron casos de tests para verificar los funcionamientos básicos del algoritmo. Se realizaron en primera instancia tests unitarios básicos, con casos desde 2 estados hasta 80 estados. Se creó un archivo de testeo unitario llamado `BranchingEquivalenceTest.java`, ubicado en `mtstools/DCS/Compositional/Tests/`, donde se implementó la lógica básica para a raíz de archivos `.lts` llamar al algoritmo de *branching bisimilarity* y luego al algoritmo de *weak equivalence* y comparar el grafo resultante obtenido. Esta comparación es la verificación empírica de la dirección que prueba el Teorema 3.4.1: *branching* nunca debe fusionar dos estados que WSOE deja separados. Para esto, primero se realizaron pruebas de todos los casos donde ambos algoritmos tenían que reducir al mismo gráfico, es decir, casos donde los algoritmos se comportaran de forma equivalente. Luego de validar estos casos, el objetivo fue encontrar un caso donde la lógica de los algoritmos difiera, es decir que por el caso de estudio del grafo elegido, ambas reducciones por la definición de equivalencia que utilizan, se comporten de forma diferente. Para ello, se implementó el caso de test presentado por van Glabbeek y Weijland [15], que es el LTS del Ejemplo 4 de la Sección 3.3 (Figura 3.4), donde se muestra que $s_2 \sim_W s_8$ pero $s_2 \not\sim_B s_8$. Este caso de test fue importante para poder asegurar que el algoritmo de *branching bisimilarity* realmente funcionara de la forma esperada, permitiendo validar de forma más robusta su implementación. A su vez, permitió desarrollar a un nivel más práctico las diferencias reales entre ambos algoritmos.

5.3.1. Integración con el algoritmo composicional

Luego de realizar los primeros tests, se revisó también la integración del nuevo algoritmo de *branching bisimilarity* con MTSA. En este contexto, se pudo detectar que en el contexto del algoritmo, una acción fluent se debería distinguir de las transiciones tau, y aun cuando esté identificada como local, no debe ser tratada como tau. Es por esto, que en la siguiente versión se agrega una validación para eliminar a las acciones fluents del conjunto de `tauLabels`. Este ajuste va en la misma dirección que el resultado del Capítulo 3: el Ejemplo 3 de la Sección 3.3 (Figura 3.3) muestra que tomar todas las acciones locales como τ hace que *branching* fusione pares que WSOE mantiene separados, y por eso el conjunto de acciones inertes se restringe a $\tau = \Upsilon_u$, que es la hipótesis bajo la cual vale el Teorema 3.4.1. El cálculo de `tauLabels` en MTSA es entonces la contraparte concreta de esa decisión teórica.

Otra diferencia a como estaba implementado el algoritmo es que en MTSA el estado de error $-1L$ es particular. En el algoritmo inicial, esto no estaba contemplado, pero luego

al realizar las primeras pruebas, lo modifiqué para que el estado de error tenga un bloque inicial propio.

A su vez, para poder integrar el algoritmo de minimización de *branching* en el algoritmo composicional, se creó la función `buildMinimisedMTSFromPartition`, que a partir de una partición que devuelve el algoritmo de *branching bisimilarity* se crea el MTS correspondiente.

5.4. Primeras optimizaciones

5.4.1. Cambio de estrategia: dos fases que alternan

Para la primera versión del algoritmo con optimizaciones (v_2), la mayor diferencia con la estructura del paper es que se implementan dos fases que alternan al mismo nivel en lugar de anidar un `while` externo sobre bunches con un `for` interno sobre splitters. Concretamente, en el Algoritmo 1 la Fase 2 se corresponde con el *while* de la Línea 5 y la Fase 1 con el *for* de las Líneas 13–29. Si bien este cambio no afecta la complejidad asintótica, mejora el entendimiento del algoritmo y ayuda a introducir los cambios que mejoran el uso de las estructuras. A su vez, también permite tener dos fases separadas y poder utilizarlas como elemento de medición para contar iteraciones y tiempos. Entonces, se modifica el algoritmo para que esté compuesto por dos fases: la primera donde se realiza la estabilización de estados y la segunda donde se refinan los bunches correspondientes.

En la primera fase, se itera sobre la `splitter list`, que almacena una estructura por cada `split` pendiente, indicando que falta estabilizar el bloque B con el conjunto de transiciones asociadas y las marcas correspondientes. Por cada `splitter`, si el bloque no está en Π_s , quiere decir que ya fue partido por un `splitter` anterior y se descarta. Esto no pasará en la primera ronda pero puede suceder en las rondas futuras. Si no, se ejecuta el `split(B, transitions, marks)` que devuelve los bloques R y U al igual que en la primera versión. Al dividir a B en R y U, hay que actualizar los `splitters` relacionados a B. En la primera versión, esto era necesario solo para el `splitter` secundario, pero como ahora el algoritmo se divide en dos fases, entonces se define la función `refineSplitters` que recorre la `splitterList` y por cada `splitter` relacionado al bloque B, se crean dos `splitters` nuevos para R y U recalculando nuevamente las marcas necesarias y manteniendo el `groupId`. Estos `splitters` son añadidos a la `splitter list` para que sean procesados a continuación. Al igual que antes, el `split` no solo afecta a los `splitters`, sino que también puede hacer que un `bunch` trivial se vuelva no trivial luego de la partición. Para encolar a estos bunches, en esta nueva versión se implementa la estructura nueva `targetStateToBunches` que es de tipo `Map<Long, Set<Set<Triple>>>` y permite almacenar por cada bloque B cuáles son los bunches que tienen alguna transición entrante a B. Esto permite no iterar sobre todo Π_t para encontrar los bunches afectados y encolarlos fácilmente. Esta estructura es el precursor de la vista `byTarget` que se describe en la Sección 5.5.2. Por último, también puede ser que haya transiciones inertes en el bloque b que al dividirlo ya no sean inertes. Este proceso se optimizó en esta versión para acercarse más a lo sugerido en el paper. Para ello, se utiliza una nueva estructura `newFrontiers` que es un `Queue<Pair<Set<Long>, Set<Long>>>`, donde cada elemento es un par que contiene un bloque en cada posición, por ejemplo `<R,U>` y `<U,R>`, indicando que se debe revisar si hay transiciones que vayan de R a U que anteriormente eran inertes pero ahora dejaron de serlo. Por cada par, se buscan estas transiciones y se registra como nuevo `bunch` en las estructuras de datos indicadas, como Π_t . Luego, al igual que en la versión inicial, se vuelve a partir el bloque

R en N y R', pero utilizando los nuevos mecanismos detallados para esta versión. El invariante que se asegura al finalizar la Fase 1, es que Π_s es estable y la `splitterList` finaliza vacía.

La segunda fase toma un bunch no trivial T de Π_t . Para T, se inicializa un mapa `Map<Pair<String, Integer>, Set<Triple<Long, String, Long>>>` llamado `slices`, donde por cada par `<a, blockID>` se almacena el conjunto de transiciones de T que tienen acción a y entran al bloque blockID. Esta es otra optimización que se agrega en esta versión, ya que antes los bloques se representaban como un `Set<Long>` pero esto representaba un costo $O(n)$ cada vez que se buscaba un bloque. En esta nueva versión, se utiliza la estructura `blockIdMap` implementada como un `IdentityHashMap<Set<Long>, Integer>`. Esta estructura asocia por cada bloque un Integer, que actúa como su blockID asociado, permitiendo obtener el número de bloque en $O(1)$. Retomando el algoritmo, T es trivial cuando hay una sola slice, por lo que en ese caso si el bunch elegido tiene una sola slice, se toma otro bunch de Π_t hasta obtener uno que no sea trivial. Al encontrar un bunch T no trivial, es decir que tiene más de un slice, al igual que en la primera versión, se eligen las `chosenTransitions` y se realiza el mismo procedimiento que el mencionado anteriormente para partir T en `chosenTransitions` y `newBunch`, también actualizando Π_t y los splitter asociados.

Entonces, la estrategia en esta nueva versión del algoritmo fue cambiar el orden de la lógica para primero procesar todos los splitters pendientes en la fase 1, luego realizar la partición de un bunch no trivial en la fase 2 y encolar nuevos splitters, para finalmente volver a iterar los splitters en la fase 2. Estas dos fases se ejecutan de manera alterna hasta que las estructuras Π_t y la `splitterList` se encuentran vacías, ya que eso asegura que todos los bloques son estables respecto a los conjuntos de transiciones, y que a su vez todas las transiciones son no triviales.

5.4.2. Implementación de nuevas estructuras

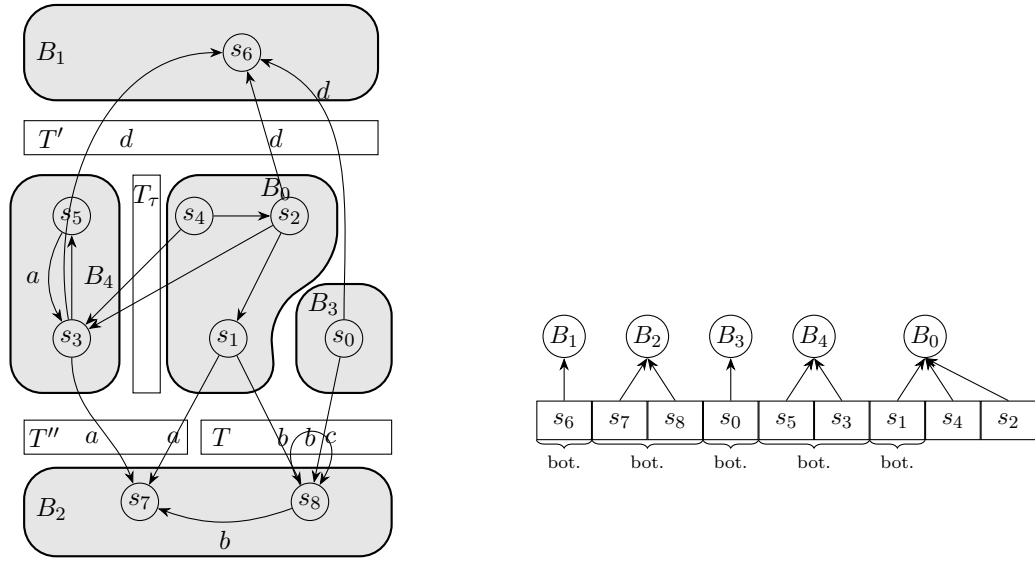
Con el objetivo de optimizar los tiempos del algoritmo, se reemplazaron varias estructuras poco ágiles de la primera versión. Por ejemplo, Π_t , antes implementada como un `List<Set<Triple>>`, ahora fue implementada como un `Set<Set<Triple>>` con `IdentityHashMap`. A su vez, en la clase `Splitter` también se realizaron varias modificaciones. Previamente, las marcas se marcaban en un mapa asociando el splitter con las marcas, ahora en cambio se implementa un campo `marks` dentro de la propia clase. Por otro lado, en la primera versión del algoritmo se explicó que para detectar cuando un splitter primario tenía un splitter secundario, se buscaba directamente el siguiente en la fila de splitters, asumiendo una adyacencia. Eso era válido en esa versión, pero ahora al tener dos fases distintas, se necesitó implementar dentro de la clase `Splitter` el campo `groupId`, marcando que si dos splitters tienen el mismo `groupId` están emparejados.

5.5. Alcanzando la complejidad teórica

Luego de realizar las primeras optimizaciones mencionadas y validar que el algoritmo funcionara correctamente, solo quedaba implementar dos cambios claves para alcanzar la complejidad teórica de $O(m \log n)$ analizada en la Sección 4.5. El argumento de esa sección se apoya en el principio de *procesar la mitad más chica*, que solo cierra si las operaciones sobre bloques y bunches se resuelven en tiempo constante; por eso hacen falta estructuras de datos específicas. En particular, el paper menciona dos estructuras claves en la sección

“detalles de implementación”. Por un lado, detalla la implementación de estados agrupados por bloques con la utilización de una estructura llamada “refinable partition”. Por otro lado, se menciona la implementación de una estructura para almacenar los bunches de transiciones llamada “Linked Refinable Partitions” a la que en el código final para esta tesis se decidió nombrar como “LinkedTransitionPartitions”. Por último, en esta nueva versión (v_3) se implementan también los cambios necesarios para poder ejecutar la función `split` como las dos corrutinas del Algoritmo 2.

5.5.1. Refinable Partitions



(a) Un ejemplo de LTS y su partición correspondiente. Las transiciones no etiquetadas deben asumirse como interres, es decir τ -transitions.

(b) Estructura Refinable Partition para el LTS de ejemplo.

Fig. 5.1: Una refinable partition para un LTS de ejemplo. (a) El LTS y su partición en cinco bloques; las transiciones sin etiqueta se asumen τ (inertes). (b) El array maestro `elements`: cada bloque es un slice contiguo, y dentro de cada slice los estados bottom (marcados bot.) se ubican primero.

Las refinable partitions son una estructura de datos que reemplaza la forma de almacenar a Π_s como una lista `List<Set<Long>>`, la estructura `stateToBlockMap` y el id de bloque `blockIdMap` de las primeras versiones. La idea principal es que todos los estados viven en un único array `elements`. Cada bloque es un slice contiguo [`start..end`] de ese array. Por lo tanto, al separar un bloque no se necesita copiar nuevamente la estructura, sino que se reordena el array con swaps $O(1)$ y se agrega un nuevo slice de bloque. La Figura 5.1 (a) muestra un ejemplo de un LTS partido en 5 bloques, y la Figura 5.1 (b) hace un *snapshot* de cómo se vería la estructura de Refinable Partition para el LTS de ejemplo, marcando también los slices contiguos de cada bloque. A su vez, dentro de cada bloque hay tres rangos disjuntos de estados, delimitados por dos punteros `bottomEnd` y `rDestEnd`. Esto se ve de forma poco clara pero introductoria en la Figura 5.1 (b), donde se puede observar que los estados no-bottom de B_0 y B_4 están agrupados de forma separada dentro de cada bloque, luego de los estados bottom, marcados como “bot.”. Una refinable

partition tiene tres divisiones: primero en $[start..bottomEnd)$, se posicionan los estados bottom. Luego, en $[bottomEnd..rDestEnd)$, se almacenarán durante el algoritmo del split, los estados no-bottom que no van a estar en R . Por último, posicionado al final de la estructura, en $[rDestEnd..end)$, se encontrarán los estados destinados a R . Se mantiene siempre el invariante $start \leq bottomEnd \leq rDestEnd \leq end$. Los estados bottom que la estructura mantiene agrupados al comienzo de cada bloque son el conjunto $Bottom(B)$ definido en la Sección 4.1.

La implementación de refinable partition se compone de dos clases: `RefinablePartition` y `Block`. La Figura 5.2 detalla la clase `Block`, compuesta por el campo de `start`, `end`, `bottomEnd`, `rDestEnd`, el `id` y si el bloque sigue activo, con `alive`. A su vez, expone también el método `size`.

```
public static final class Block {
    int start;
    int end;
    int bottomEnd;
    int rDestEnd;
    public final int id;
    boolean alive;

    public int size() { return end - start; }
}
```

Fig. 5.2: Definición de la clase `Block`

A su vez, se define la clase “`RefinablePartition`”, detallada en la Figura 5.3. Se exponen los métodos de construcción, `RefinablePartition(Collection<Long> universe)` y `void seedFromInitialBlocks(List<Set<Long>> initialBlocks)`. Luego, los métodos de lookup, donde se pueden notar claramente las mejoras en los tiempos de acceso en esta estructura. `Block blockOf(long s)` devuelve el bloque al que pertenece el estado s , en $O(1)$. `Iterable<Long> states(Block b)` retorna los estados del bloque b , en $O(|b|)$. `Iterable<Long> bottoms(Block b)` retorna los estados bottoms del bloque b , en $O(|cantestadosbottomdeb|)$. `Iterable<Long> rDestStates(Block b)` retorna todos los estados destinados a R del bloque B , utilizando el puntero ya mencionado, $rDestEnd$, en $O(|b.rDestCount|)$. Por último, `List<Block> liveBlocks()` retorna todos los bloques existentes en la `RefinablePartition`. Luego, se exponen también los métodos necesarios para realizar un split.

La Figura 5.4 sigue cómo va mutando la estructura durante un split, paso a paso. Observar este proceso ayudará a explicar cómo funcionan los métodos `addToR`, `clearR` y `splitOffR` que son el corazón de la `RefinablePartition`. Observemos el slice de la estructura `RefinablePartition` que representa al bloque B . Para ejecutar las operaciones, lo único que se mueve son los estados dentro del slice de B (a través de swaps) y los punteros `bottomEnd` y `rDestEnd`. En ningún momento se copian estados a una estructura nueva ni a otro slice. A continuación, el paso a paso de un split.

(a) `clearR(B)` (Figura 5.4). Por default, un bloque comienza con `rDestEnd = end`, por lo que la partición destinada a R queda vacía. El bloque contiene sólo dos rangos visibles: los estados bottom $[start..bottomEnd)$ y el resto no-bottom. La función `clearR` asegura que la partición de R comience vacía.

(b) `addToR(s) sobre un no-bottom` (Figura 5.5). Si se debe agregar un estado s no-bottom a R , se hace un swap del estado contra la posición `rDestEnd-1` y se retrocede

```

final class RefinablePartition {
    RefinablePartition(Collection<Long> universe);
    void seedFromInitialBlocks(List<Set<Long>> initialBlocks);

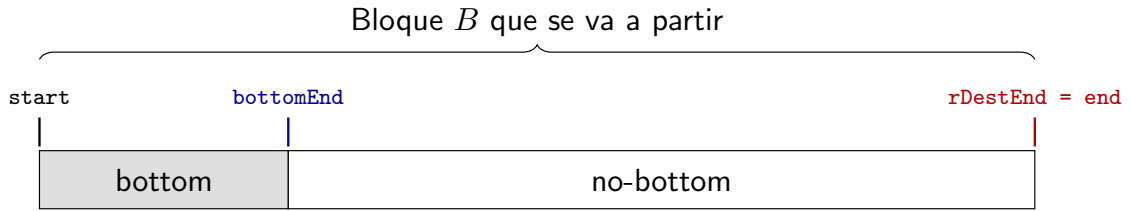
    Block blockOf(long s);
    Iterable<Long> states(Block b);
    Iterable<Long> bottoms(Block b);
    Iterable<Long> rDestStates(Block b);
    List<Block> liveBlocks();

    void addToR(long s);
    void clearR(Block b);
    Pair<Block, Block> splitOffR(Block b);

    void markAsBottom(long s);
    void resetBottoms(Block b);
}

```

Fig. 5.3: Definición de la clase RefinablePartition

Fig. 5.4: Paso (a): estado inicial del bloque B (`clearR`).

`rDestEnd` en uno. La partición R crece de a un estado por llamada, sin tocar los estados `bottom`. $O(1)$ ya que solo se debe realizar el swap y sumar o restar a los punteros.

(c) **addToR(b) sobre un bottom** (Figura 5.6). Si el estado era `bottom`, primero se lo saca de ahí con un swap contra `bottomEnd-1` y se retrocede `bottomEnd`. De esta forma, el estado queda incluido en la partición R al igual que en (b). $O(1)$ ya que solo se debe realizar el swap y sumar o restar a los punteros.

(d) **Estado tras repetir addToR** (Figura 5.7). Una vez agregados todos los estados de R , la partición `[rDestEnd..end)` contiene los estados indicados y el slice de B quedó ordenado en tres particiones.

(e) **splitOffR(B)** (Figura 5.8). Se corta el slice en `rDestEnd`: la primera partición `[start..rDestEnd)` pasa a ser `uBlock` y la segunda partición `[rDestEnd..end)` pasa a ser `rBlock`, cada uno con su nuevo id. El viejo B se marca como muerto y se reasigna `blockOfElement` únicamente en las posiciones del sufijo para que apunten a `rBlock`. Crear los bloques en sí es $O(1)$, ya que solo implica crear una nueva estructura `Block` con los punteros de `start` y `end` y no copiar todos los elementos de R ; la reasignación de `blockOfElement` sobre el sufijo es la única pasada lineal del split, y cuesta $O(|rBlock|)$. Esto reemplaza a la copia de dos `HashSet` de las versiones anteriores.

(f) **findBottomStates** (Figura 5.9). Como al hacer el split los bottoms del bloque viejo ya no valen, se recomputan utilizando `resetBottoms`, que realiza `bottomEnd = start` y `markAsBottom` que realiza un swap y mueve el puntero `bottomEnd` según corresponda.

Para concluir, también se reemplazó la inicialización de Π_s , que en las anteriores versiones era implementado como una lista `List<Set<Long>>`, por las llamadas a los constructores de `RefinablePartitions`, tal como se ve en la Figura 5.10. Así, se logra bajar la

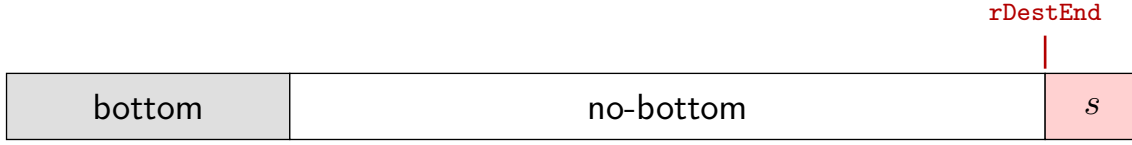


Fig. 5.5: Paso (b): addToR sobre un estado no-bottom.

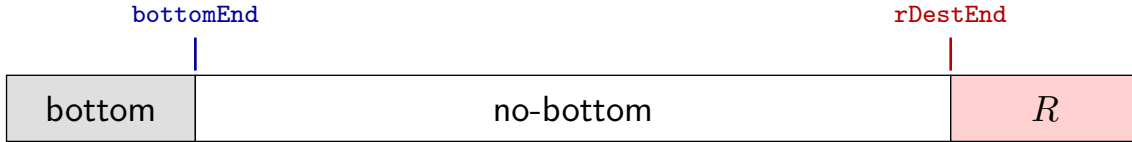


Fig. 5.6: Paso (c): addToR sobre un estado bottom.

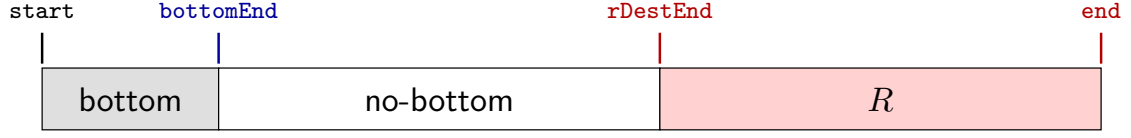
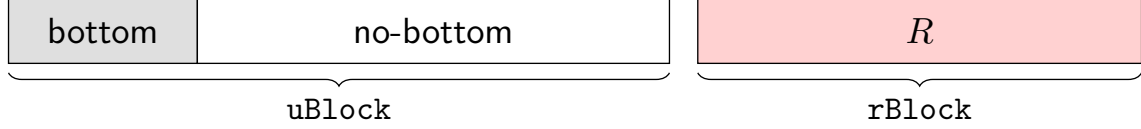
complejidad de varias operaciones frecuentes de Π_s . Por ejemplo, cuando previamente se quería ver si Π_s contenía un bloque B, se debía recorrer toda la lista de bloques, es decir que tenía complejidad $O(|\Pi_s|)$. Con la nueva estructura, esto se realiza en $O(1)$, simplemente viendo si el bloque B tiene el campo `alive` en `true`. A su vez, para encontrar los bottom states de un bloque B, se iteraba por todos los estados hasta encontrar los bottom. Ahora, se pueden encontrar todos los bottoms directamente en la partición `[start..bottomEnd)`.

5.5.2. Linked Transition Partitions

En el paper de Groote et al. [20] también se sugiere la implementación de Π_t como “linked refinable partitions”. Así, la idea es reemplazar los bunches de Π_t que en las primeras versiones estaban representados como `Set<Triple<Long, String, Long>>`. Para ello, se define la clase `Transition` tal como muestra la Figura 5.11, donde cada transición es un objeto que almacena el estado de origen, la acción y el estado destino. A su vez, se almacena el bunch de Π_t que la contiene, y el block-bunch al que pertenece, es decir el subgrupo que indica dentro del bunch al que pertenece, de qué estado de origen es. Por último, se implementa el campo `untested`, que es el contador `untested[t]` descrito en la Sección 4.4 y usado en la Línea 9 del Algoritmo 2, necesario para la implementación de las corrutinas en el `split`.

Luego, se define la clase `LinkedTransitionPartitions`, tal como se ve en la Figura 5.12. Se hace referencia a Π_s ya que se debe consultar en diversos métodos, debido a que los bunches dependen de los bloques de estados. El campo `globalUnstable` es la lista de todas las block-bunch-slices inestables, en el sentido de la Sección 4.1, es decir, la cola de trabajo pendiente de la Fase 2: implementada como una lista con position tracking, permite agregar y quitar elementos en $O(1)$. Por último, `nextBunchId` y `nextBlockBunchId` son contadores que asignan un identificador único a cada `BunchSlice` y `BlockBunchSlice` en el momento de su creación. A su vez, está la lista `allTransitions`, que es la lista que contiene todas las transiciones del LTS. Esta es la lista principal sobre la cual se construyen las cuatro vistas de las transiciones descritas en el paper que permiten sostener la complejidad deseada.

Por un lado, esta la vista por bunch `liveBunches`. Esta vista se implementa como una lista de bunch slices `List<BunchSlice>`, donde la clase `BunchSlice` agrupa las transiciones de un mismo bunch de Π_t . Esta es la vista que implementa directamente la partición Π_t , y reemplaza al `IdentityHashMap` que en las versiones anteriores representaba el conjunto

Fig. 5.7: Paso (d): estado del bloque tras repetir `addToR`.Fig. 5.8: Paso (e): `splitOffR` divide el bloque en `uBlock` y `rBlock`.

de bunches. La Figura 5.13 muestra cómo queda la estructura sobre el ejemplo de la Figura 5.1. A diferencia del paper, esta vista no mantiene de forma explícita los *action-block-slices* $T_{a \rightarrow B'}$ definidos en la Sección 4.1 (los subconjuntos de transiciones de un mismo bunch con igual acción y bloque destino), sino que estos se calculan momentáneamente solo cuando hace falta refinar un bunch, en el método `peelSlice`, que es también donde se evalúa la condición $\#aB'(T) > 1$ de la Línea 5 del Algoritmo 1.

Luego, la segunda vista es `slicesByBlock`, que se implementa como un mapa `Map<Block, List<BlockBunchSlice>>`, donde cada bloque de Π_s tiene asociada una lista de todos sus *block-bunch-slices* $T_{B \rightarrow}$, definidos en la Sección 4.1, que serían las transiciones que se originan desde el bloque dentro de cada bunch. Este mapa permite recuperar en $O(1)$ todas las *block-bunch-slices* de un bloque, lo cual es utilizado cuando se parte un bloque, ya que se debe redistribuir las *block-bunch-slices* del bloque viejo entre los dos bloques nuevos. La Figura 5.14 muestra la estructura de `slicesByBlock` para el ejemplo presentado anteriormente.

La tercera vista mencionada agrupa las transiciones por su estado de origen. Se implementa como un mapa de tipo `Map<Long, List<Transition>>` llamado `bySource` y se expone con el método `outgoing(s)`, que devuelve todas las transiciones salientes del estado s . Esta vista es *estática*, ya que como la lista de transiciones salientes de un estado se fija según el LTS de entrada y no varía, la estructura no se modifica en ningún paso del algoritmo. Se usa de referencia para poder obtener todas las transiciones salientes de un estado, por ejemplo, durante el `split`. La Figura 5.15 muestra la estructura, donde cada estado de origen tiene asociada la lista de sus transiciones salientes.

Por último, la cuarta vista es la inversa de la anterior y agrupa las transiciones por su estado destino, tal como muestra la Figura 5.16. Se implementa como un mapa de transiciones `Map<Long, List<Transition>>` denotado `byTarget` y se expone con el método `incoming(s)`, que devuelve las transiciones entrantes al estado s . Por el mismo motivo que la vista anterior, también es *estática*. Optimiza el proceso de cuando un bloque se parte, ya que se deben marcar como inestables las *block-bunch-slices* de los bunches que tenían transiciones apuntando a los estados del bloque que se separó, y `byTarget` permite acceder a ellas fácilmente recorriendo para cada estado del nuevo bloque, sus transiciones entrantes.

Es importante aclarar que las cuatro vistas no implican que haya cuatro copias de las transiciones, sino que existen cuatro agrupaciones de los mismos objetos `Transition`, ya que cada transición guarda punteros a su `BunchSlice` y a su `BlockBunchSlice`, y el

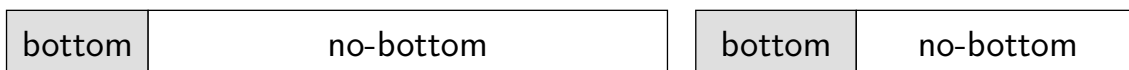


Fig. 5.9: Paso (f): `findBottomStates` recomputa los estados bottom en cada bloque nuevo.

```
RefinablePartition Pi_s = new RefinablePartition(toMinimise.getStates());
Pi_s.seedFromInitialBlocks(initialBlocks);
```

Fig. 5.10: Inicialización de Π_s como `RefinablePartition`

puntero a la propia `Transition` es lo que se relaciona en las listas de `bySource` y `byTarget`. Es por esto que el paper las llama “linked”, ya que una única instancia participa de las cuatro vistas/particiones a la vez.

5.5.3. El método `split` como corrutinas

Como se explicó anteriormente, la función `split` en el paper se idea originalmente como dos corrutinas, tal como se detalla en la Sección 4.4 y se formaliza en el Algoritmo 2, donde la corrutina de `R` comienza con los estados iniciales que deben ir en `R` y luego extiende el conjunto hacia atrás, recorriendo las transiciones inertes. A su vez, la corrutina de `U` identifica a los estados que no están en `R`, contando por cada estado cuántos son sus sucesores que todavía no están en `U`. Ejecutar estas corrutinas de forma alternada, un paso cada una (en *lockstep*), es la esencia de la complejidad del paper, tal como se analiza en el párrafo “División de bloques” de la Sección 4.5, ya que en cuanto una corrutina agrega a su conjunto más de la mitad de estados totales que tenía el bloque, se corta su ejecución. Este corte consigue que cada llamada a la función `split` pase de tener que recorrer todo el bloque en peor caso, a que pase a recorrer solo el subbloque más chico de los dos conjuntos de estados resultantes. Sin embargo, en las primeras versiones la implementación se había reducido a realizar *backwards-reachability* para `R`, es decir, no se había desarrollado la optimización sugerida en el paper. Por su importancia en la complejidad final, era esencial implementarla en nuestra versión en MTSA. Sin embargo, Java no tiene la posibilidad de implementar corrutinas de forma nativa.

Para poder subsanar esa dificultad en la implementación, se implementaron dos funciones: `rStep` que ejecuta un paso en el análisis del bloque `R` y `uStep` que ejecuta un paso en el análisis del bloque `U`. Ahora, ¿cómo representamos qué es un “paso” en las funciones? Para esta implementación, un paso en cada función es analizar el próximo estado que corresponde chequear si pertenece al subbloque de la función. Para ello, se implementaron dos estructuras de colas, una para cada función, que contienen los estados que

```
public static final class Transition {
    public final long source;
    public final String action;
    public final long target;
    BunchSlice bunch;
    BlockBunchSlice blockBunch;
    int untested;
}
```

Fig. 5.11: Definición de la clase `Transition`

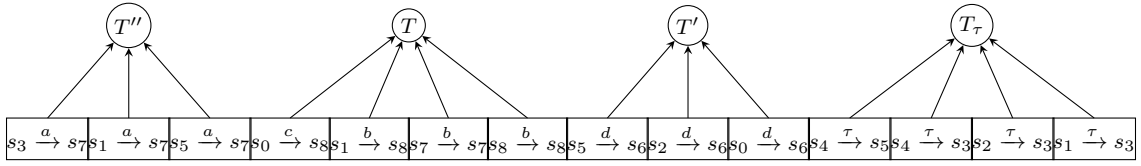
```

public final class LinkedTransitionPartitions {
    RefinablePartition Pi_s;
    List<Transition> allTransitions;
    Map<Long, List<Transition>> bySource;
    Map<Long, List<Transition>> byTarget;
    List<BunchSlice> liveBunches;
    Map<RefinablePartition.Block, List<BlockBunchSlice>> slicesByBlock;
    List<BlockBunchSlice> globalUnstable;
    Map<Triple<Long, String, Long>, Transition> transitionLookup;

    nextBunchId;
    nextBlockBunchId;
}

```

Fig. 5.12: Definición de la clase LinkedTransitionPartitions

Fig. 5.13: La estructura de liveBunches, que representa la vista de todas las transiciones ordenadas por bunches de Π_t . Se usa el ejemplo de la Figura 5.1.

faltan analizar. En cada paso, cada corrutina toma el siguiente paso y ejecuta el análisis correspondiente para ver si ese estado corresponde o no en su conjunto. A su vez, se implementan los contadores "rSize" y "uSize", que son los que materializan el corte en $\frac{1}{2} |B|$ del Algoritmo 2. Esto permite validar que cuando alguno de los dos contadores supere la mitad de los estados del bloque, se deje de ejecutar esa corrutina y no se sigan analizando estados para ese subbloque, haciendo que solo finalice la corrutina del subbloque más chico y luego se calcule el conjunto final de estados para cada subbloque, tal como se detalla en el paper y tal como fue explicado en la sección previa.

Si bien esta implementación no mantiene de manera completamente fiel el detalle del paper, permite que repliquemos la cota de $O(m \log n)$ de la Sección 4.5 dentro de las limitaciones del lenguaje utilizado.

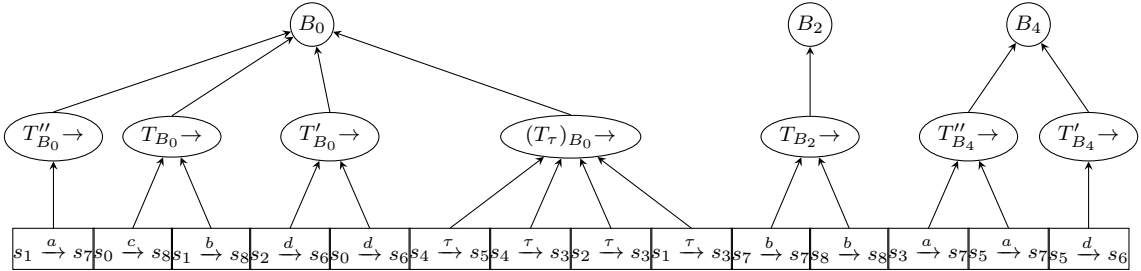


Fig. 5.14: La estructura de slicesByBlock, mostrando la vista de las transiciones agrupadas por block-bunch-slice.

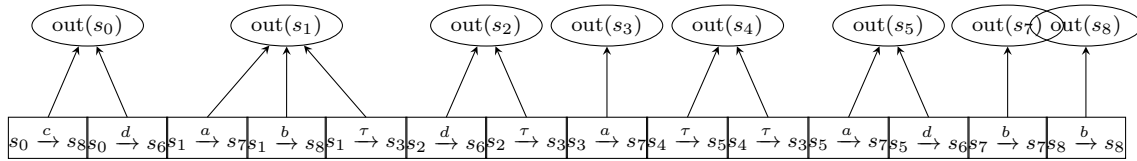


Fig. 5.15: La estructura **bySource**, que representa la vista de transiciones ordenadas por estado de origen.

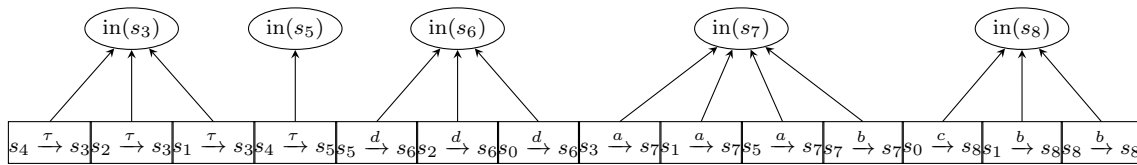


Fig. 5.16: La estructura **byTarget**, mostrando las transiciones agrupadas por estado destino.

6. VALIDACIÓN Y EXPERIMENTACIÓN

6.1. Metodología

Luego de finalizar la implementación, continuamos con la evaluación del método. Por un lado, se quería evaluar la *correctitud* de la versión final del código del algoritmo en MTSA, y por el otro lado se quería evaluar el *rendimiento* del mismo, para luego poder compararlo con el algoritmo de WSOE previamente implementado en MTSA.

Al comenzar a realizar estas experimentaciones, se intentó reemplazar el llamado al algoritmo de WSOE dentro del proceso de síntesis composicional, por el llamado al recién implementado algoritmo *Branching bisimilarity*. Sin embargo, a medida que se comenzaron a realizar estas experimentaciones, la obtención de resultados se hacía cada vez más costosa y poco escalable, ya que el tiempo que tardaba en correr el proceso de síntesis completo era muy largo, lo que no nos permitía tener flexibilidad a la hora de afinar y mejorar los resultados obtenidos. Por ello, se aplicó una metodología de captura de las instancias de minimización, de forma que se corrió una vez el algoritmo de síntesis composicional completo con los conjuntos de casos de prueba ya armados por Gagliardi et al. [10] y en el contexto de este proceso, se almacenó en diferentes archivos todos los parámetros que eran introducidos en el llamado al algoritmo de minimización. Esto nos permitió extraer 2212 instancias de casos de LTS a minimizar, con las que luego pudimos validar y experimentar de forma más flexible y eficiente con las diferentes versiones de nuestra implementación del algoritmo de *Branching bisimilarity*.

Cada instancia capturada queda representada por un par de archivos. El primer archivo, de tipo `.lts`, contiene el autómata a minimizar en el formato FSP que interpreta MTSA, es decir, sus estados y sus transiciones etiquetadas. El segundo archivo almacena en formato JSON el resto de los parámetros que la llamada a la minimización recibe junto con el LTS, lo que incluye principalmente el alfabeto local, las etiquetas controlables y las locales. Luego, cada par `.lts/.json` contiene toda la información necesaria para reconstruir exactamente el problema de minimización de forma aislada, sin necesidad de reejecutar la síntesis composicional que lo generó.

Cada corrida de experimentación para un algoritmo de minimización registra sus resultados en un archivo CSV en el que se agrega una fila por cada instancia, versión del algoritmo y repetición, para permitir correr varias veces cada instancia y luego tomar la mediana de los valores resultantes. Además del tamaño del resultado (cantidad de estados, de transiciones y de transiciones *tau* del LTS minimizado), cada fila guarda el tiempo de ejecución y el uso de memoria. Para obtener mediciones más confiables, cada instancia se ejecutó 3 veces, y se registró también el estado final de la corrida (éxito, error o *timeout*).

Por último, se corrió la misma para tres versiones del nuevo algoritmo de *Branching bisimilarity* implementado en MTSA. En primer lugar, la v1 que es la primera implementación *naive* del algoritmo, presentada en la Sección 5.2. Luego la v2, que es la presentada en la Sección 5.4, con las primeras optimizaciones. Luego, la v3, que es la versión final del código con todas las optimizaciones ya implementadas, tal como se detalla en la Sección 5.5. Por último, se realizó a su vez una corrida de la experimentación del algoritmo de WSOE para poder utilizar sus resultados en comparación.

Los experimentos fueron ejecutados en una computadora con procesador Intel i7-7700

CPU @ 3.60GHz, con 8GB de RAM.

6.2. Validación de correctitud

Con el objetivo de validar la correctitud del algoritmo, se buscó en primer lugar tomar instancias de diversos tamaños y de distintos casos, asegurando construcciones variadas, y se utilizaron a estas instancias para comparar el resultado obtenido con el algoritmo de *branching* con el resultado obtenido por la herramienta mCRL2 [23], cuya implementación de *branching bisimulation* fue desarrollada por los propios Groote y Jansen a partir del algoritmo [20] descrito en el Capítulo 4. Esto la convierte en un oráculo adecuado para la validación, ya que implementa exactamente la misma noción de equivalencia y el mismo algoritmo que nuestra implementación en MTSA. Concretamente, para cada instancia se corrió el mismo LTS de entrada por ambas implementaciones y se comparó el LTS minimizado que devolvía cada una, verificando que el resultado obtenido fuera el mismo. Esto nos permitió validar la correctitud de nuestro algoritmo como primer paso.

Luego, se corrió también una validación de correctitud más fuerte, corriendo el algoritmo de WSOE y el de *Branching* y validando que para cada instancia, nunca debía suceder que *Branching* junte dos estados que WSOE dejaba separados, ya que eso implicaría un bug en el código desarrollado, tal como se detalló en el Teorema 3.4.1.

Luego de validar la correctitud y asegurar que el algoritmo estaba funcionando en términos teóricos tal como se esperaba, se procedió con la experimentación.

6.3. Resultados

Tal como se detalló anteriormente, las instancias fueron capturadas replicando los casos de prueba del procesamiento de síntesis de controladores. En este contexto, la mayor parte de las ocasiones se busca minimizar grafos lo más pequeños posibles, y son pocas las ocasiones donde se debe minimizar un grafo más grande. Es por eso que el 14,6 % de las instancias (322 de las 2212) tienen más de 100 estados. Los experimentos resultan más interesantes de analizar en casos donde los grafos son más grandes ya que ahí podremos ver mejor la materialización de la complejidad teórica. A su vez, la proporción entre estados y transiciones es desigual, ya que la mayoría de las instancias tiene más del doble de transiciones que de estados, lo que resulta en que más del 27 % de las instancias analizadas supera las 200 transiciones. Esto nos dará un enfoque particularmente interesante de analizar, ya que la complejidad de nuestro algoritmo está atada también al número de transiciones del LTS a minimizar.

6.3.1. Tamaño de la reducción final

En este contexto, nos interesaba analizar cómo impactaban para estas instancias el resultado final de la minimización realizada. En ese sentido, luego de correr la experimentación para WSOE y para la versión final de *branching* (v3), pudimos ver que en el 83 % de los casos analizados, el resultado de la minimización de WSOE fue el mismo que el obtenido por el algoritmo de *branching bisimilarity*. Es decir, solo en un 17 % de los casos sucedió el caso ejemplificado en el Ejemplo 4 de la Sección 3.3 (Figura 3.4). Este resultado es útil para demostrar que en la mayoría de los casos, a pesar de la diferencia en la equivalencia teórica entre ambos algoritmos, los casos donde esta diferencia se materializa en la experimentación es la minoría.

6.3.2. Tiempos

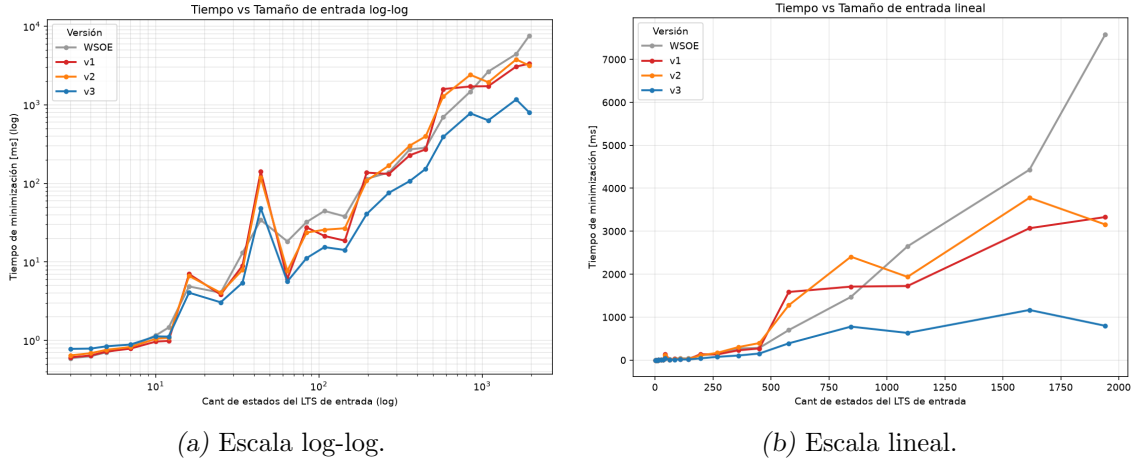


Fig. 6.1: Tiempo de minimización en función de la cantidad de estados del LTS de entrada.

Las Figuras 6.1a y 6.1b muestran el tiempo de minimización en función de la cantidad de estados iniciales, la primera en escala log-log y la segunda en escala lineal. En la escala log-log se ve que para instancias chicas el tiempo se ve dominado por el costo fijo de preparar la ejecución y no el algoritmo en sí, por lo que ahí no hay diferencias entre WSOE y las versiones de *branching*. Recién a partir de los $n \approx 100$ estados las curvas se separan y se vuelve visible el comportamiento asintótico de cada algoritmo, con v3 quedando consistentemente por debajo del resto. La escala lineal muestra de forma evidente la magnitud de esa diferencia en los grafos grandes, ya que mientras WSOE crece de forma prácticamente lineal con el tamaño y llega a tardar del orden de 7600 ms en las instancias más grandes, v3 se mantiene por debajo de los 1000 ms. Las versiones v1 y v2 siguen de cerca a WSOE, por momentos comportándose brevemente mejor. Esto demuestra que las optimizaciones finales sugeridas por el paper fueron claves para materializar la mejora asintótica en la complejidad del algoritmo.

Como se mencionó al comienzo de este capítulo, la mayoría de las instancias tiene muchas más transiciones que estados, y la complejidad teórica del algoritmo de *branching* depende justamente de la cantidad de transiciones. Por eso resulta interesante analizar el tiempo también en función de las transiciones. Las Figuras 6.2a y 6.2b muestran el tiempo de minimización contra la cantidad total de transiciones m del LTS de entrada (izquierda) y contra la cantidad de transiciones τ , ambas en escala log-log. En la Figura 6.2a se ve que v3 se mantiene por debajo de v2 y de WSOE a lo largo de casi todas las instancias, y que el tiempo de las versiones de *branching* queda correlacionado con m . La Figura 6.2b analiza particularmente las transiciones τ (excluyendo las instancias sin ninguna transición τ). Esto resulta interesante ya que por la naturaleza del algoritmo de *branching*, el algoritmo se debería comportar mejor para casos donde hay más estados posibles de minimizar, ya que allí se deben realizar menos iteraciones sobre las particiones. Este gráfico muestra justamente que la ventaja del algoritmo de *branching* se expone justamente sobre las instancias con más transiciones τ . Por último, nuevamente podemos ver como se muestra la mejora sobre los tiempos al implementar las optimizaciones mencionadas en el paper, que fueron claves para alcanzar y mejorar los tiempos de WSOE.

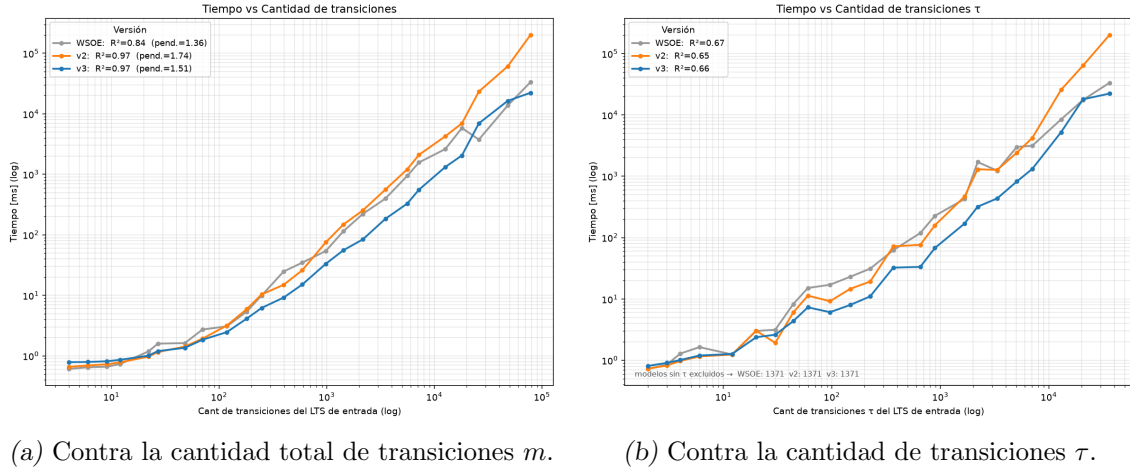
(a) Contra la cantidad total de transiciones m .(b) Contra la cantidad de transiciones τ .

Fig. 6.2: Tiempo de minimización en función de la cantidad de transiciones del LTS de entrada (mediana por bin de tamaño, escala log-log).

6.3.3. Memoria

Según el paper de Jansen, la utilización del espacio de memoria del algoritmo de *Branching bisimilarity* es menor a varios algoritmos previos con los que los autores la comparan. Sin embargo, a partir de las experimentaciones realizadas obtuvimos el resultado graficado en la Figura 6.3a. Como se muestra, todas las versiones de *Branching* utilizaron, en promedio, más memoria que la versión de WSOE. Para analizar este resultado, usamos la memoria *asignada* por cada minimización, $\Delta = \text{MemAfter} - \text{MemBefore}$. Para la mayoría de las instancias, este número se mantiene estable para todas las repeticiones de las corridas del algoritmo analizado, lo que nos da la pauta de que es una medida confiable.

Analizando la Figura 6.3a podemos ver que la v2 utiliza en casi todos los casos más memoria que las demás versiones de *Branching* y de WSOE. Analizando esta diferencia, se puede suponer que esto se debe a que la v2 es la que menos estructuras nativas de Java utiliza. Esta versión utiliza varios índices propios para introducir las primeras optimizaciones, como el `IdentityHashMap targetStateToBunches` y la `IdentityQueue`, manteniendo a su vez la implementación de v1 de los bloques con `Set<Long>` y para los bunches, `Set<Triple>`. Estos índices duplican las referencias a los estados y a las transiciones, y por eso aumenta tanto el uso de la memoria. A su vez, v1 al no tener estas referencias, es más liviana. Por último, v3 vuelve a ser más liviana porque al introducir las clases de `RefinablePartition` y `LinkedTransitionPartitions`, el costo de los índices se absorbe dentro de la misma estructura.

Sin embargo, la comparación que más nos interesa es por qué el uso de memoria de v3 es mayor que el de WSOE. Analizando ambos algoritmos, podemos ver que WSOE utiliza una única estructura de datos para la partición de estados, por lo que no se debe mantener también nuevas estructuras para las particiones de transiciones, sino que eso se recalcula constantemente a partir de cada unificación de estados. Esto implica mayor tiempo de procesamiento pero también menos uso de memoria. Por último, también se debe considerar que el algoritmo de *Branching* declara varias estructuras de datos nuevas, propias del algoritmo, mientras que WSOE utiliza tipos de datos nativos para las estructuras. Esto resulta en que la recolección y optimización de memoria de Java sea más eficiente para WSOE que para *Branching*.

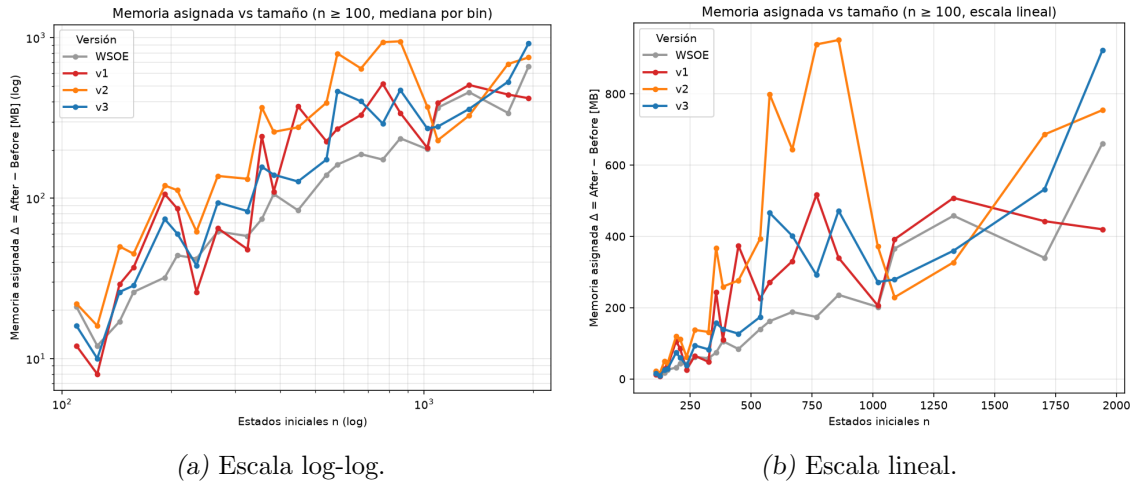


Fig. 6.3: Memoria asignada por la minimización ($\Delta = \text{MemAfter} - \text{MemBefore}$, mediana por bin de tamaño) en función de la cantidad de estados del LTS de entrada, para instancias con $n \geq 100$.

7. CONCLUSIONES

Esta tesis se propuso mejorar la escalabilidad de la síntesis composicional de controladores atacando su cuello de botella: el paso de minimización que se ejecuta una vez por iteración y que hasta ahora se resolvía calculando *Weak Synthesis Observational Equivalence* (WSOE) con un algoritmo de complejidad $O(nm)$. La propuesta fue reemplazarlo por *branching bisimilarity*, para la cual existe el algoritmo $O(m \log n)$ de Groote, Jansen, Keiren y Wijs [20].

El primer resultado del trabajo fue descubrir que el reemplazo no es directo. Las dos equivalencias no coinciden, y lo que hace falta garantizar es una dirección concreta: que *branching* nunca fusione un par de estados que WSOE mantiene separado, ya que eso significaría reducir de más y potencialmente perder soluciones del problema de control. La hipótesis natural —tomar como acciones internas a todas las acciones locales— resulta incorrecta, y el Ejemplo 3 de la Sección 3.3 exhibe un contraejemplo concreto: al ocultar una acción controlable local, *branching* fusiona un par que WSOE distingue. Restringiendo las acciones internas a las acciones locales *no controlables* ($\tau = \Upsilon_u$) la correspondencia se restablece, y el Teorema 3.4.1 demuestra que en ese caso vale $\sim_B \subseteq \sim_W$. La inclusión además es estricta, como muestra el Ejemplo 4 adaptado de van Glabbeek y Weijland [15].

Con ese resultado como base, los objetivos alcanzados fueron los siguientes:

- Caracterizar formalmente la relación entre WSOE y *branching bisimilarity* en el contexto de la síntesis composicional, identificando la condición sobre el conjunto de acciones internas bajo la cual el reemplazo es correcto.
- Implementar el algoritmo $O(m \log n)$ de Groote et al. dentro de MTSA [22] e integrarlo al flujo composicional en reemplazo de la llamada existente a WSOE. La implementación atravesó tres iteraciones: una primera versión funcional, una segunda con optimizaciones parciales, y una tercera que incorpora las estructuras `RefinablePartition` y `LinkedTransitionPartitions` necesarias para alcanzar efectivamente la cota teórica.
- Desarrollar una metodología de evaluación basada en la captura de 2212 instancias reales de minimización, extraídas instrumentando una corrida completa de la síntesis composicional sobre el conjunto de casos de prueba existente. Esto permitió iterar sobre la implementación sin pagar el costo de reejecutar la síntesis en cada experimento.
- Validar la correctitud de la implementación contra dos oráculos independientes: la implementación de referencia de mCRL2 [23], escrita por los propios autores del algoritmo, y el resultado del algoritmo de WSOE, verificando sobre cada instancia que nunca ocurriera la fusión que el Teorema 3.4.1 prohíbe.
- Evaluar empíricamente el rendimiento. La versión final se mantiene por debajo de los 1000 ms en las instancias más grandes, donde WSOE alcanza los 7600 ms, y la ventaja se acentúa sobre los modelos con mayor cantidad de transiciones internas.

En el 83 % de las instancias ambas equivalencias producen exactamente la misma reducción.

El trabajo también deja identificadas algunas limitaciones:

- **Uso de memoria.** Contra lo que sugiere el análisis del paper original, todas las versiones de *branching* consumieron en promedio más memoria que WSOE. La razón es estructural: WSOE mantiene una única partición de estados y recalcula el resto en cada paso, mientras que el algoritmo de Groote et al. mantiene simultáneamente particiones de estados y de transiciones con sus índices asociados. Es un intercambio explícito de memoria por tiempo.
- **La mejora sólo se materializa en instancias grandes.** Por debajo de los $n \approx 100$ estados el tiempo está dominado por costos fijos y no hay diferencia apreciable entre los algoritmos. Como la mayoría de las instancias que genera la síntesis composicional son chicas, el impacto sobre el tiempo total del método depende de cuántas instancias grandes aparezcan en cada problema.
- **Las optimizaciones no son opcionales.** Las versiones v1 y v2, que implementan el mismo algoritmo pero sin las estructuras de datos que el paper especifica, no superan a WSOE. Recién la v3 materializa la mejora asintótica. Esto confirma que la cota $O(m \log n)$ es una propiedad del algoritmo *junto con* sus estructuras, y no del esquema de refinamiento en abstracto.

Como trabajo a futuro, presentamos las siguientes ideas:

- Reducir el consumo de memoria de la implementación, explorando representaciones más compactas de las particiones de transiciones o el uso de arreglos primitivos en lugar de las estructuras de colecciones de Java. (aca poner el nuevo paper)
- Rep. simbolica

8. APÉNDICE

Bibliografía

- [1] P. J. Ramadge and W. M. Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987.
- [2] A. Pnueli and R. Rosner. On the Synthesis of a Reactive Module. In *Proc. of the Symp. on Principles of Programming Languages (POPL)*, pages 179–190, 1989.
- [3] Dana Nau, Malik Ghallab, and Paolo Traverso. *Automated Planning: Theory & Practice*. Morgan Kaufmann Publishers Inc., 2004.
- [4] N. Piterman, A. Pnueli, and Y. Sa’ar. Synthesis of Reactive(1) Designs. In *Verification, Model Checking, and Abstract Interpretation (VMCAI)*, LNCS 3855, pages 364–380, 2006.
- [5] N. D’Ippolito, V. Braberman, N. Piterman, and S. Uchitel. Synthesis of Live Behaviour Models. In *Proc. of the ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*, pages 77–88, 2010.
- [6] S. A. Zudaire, M. Garrett, and S. Uchitel. Iterator-based temporal logic task planning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11472–11478, 2020.
- [7] D. Ciolek. Síntesis dirigida de controladores para sistemas de eventos discretos. Tesis doctoral, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA, 2018.
- [8] M. Durán, F. Zanollo. Síntesis dirigida de controladores para requerimientos de tipo non-blocking. Tesis de Licenciatura, Laboratorio de Fundamentos y Herramientas para la Ingeniería del Software (LaFHIS), FCEyN, UBA, 2021.
- [9] J. Braier, D. Ciolek, M. Durán, F. Zanollo, V. Braberman, N. D’Ippolito, S. Uchitel, N. Pazos. *On-the-fly Informed Search of Non-blocking Directed Controllers*.
- [10] H. Gagliardi, V. Braberman, S. Uchitel. Scaling GR(1) Synthesis via a Compositional Framework for LTL Discrete Event Control. In *Computer Aided Verification (CAV)*, pages 201–223. Springer Nature Switzerland, 2025. https://doi.org/10.1007/978-3-031-98685-7_10
- [11] S. Mohajerani, R. Malik, M. Fabian. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Transactions on Automatic Control*, 59(1):150–162, 2014.
- [12] S. Mohajerani, R. Malik, M. Fabian. An Algorithm for Compositional Nonblocking Verification Using Special Events. *Chalmers University of Technology / University of Waikato*, 2012.
- [13] H. Flordal, R. Malik, M. Fabian, K. Åkesson. Compositional Synthesis of Maximally Permissive Supervisors Using Supervision Equivalence. *Discrete Event Dynamic Systems*, 17(4):475–504, 2007.

- [14] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [15] R. J. van Glabbeek, W. P. Weijland. Branching Time and Abstraction in Bisimulation Semantics. *Journal of the ACM*, 43(3):555–600, 1996.
- [16] J. E. Hopcroft. An $n \log n$ Algorithm for Minimizing States in a Finite Automaton. In *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.
- [17] R. Paige, R. E. Tarjan. Three Partition Refinement Algorithms. *SIAM Journal on Computing*, 16(6):973–989, 1987.
- [18] P. C. Kanellakis, S. A. Smolka. CCS Expressions, Finite State Processes, and Three Problems of Equivalence. *Information and Computation*, 86(1):43–68, 1990.
- [19] J. F. Groote, F. Vaandrager. An Efficient Algorithm for Branching Bisimulation and Stuttering Equivalence. In *Automata, Languages and Programming (ICALP)*, LNCS 443, pages 626–638, 1990.
- [20] J. F. Groote, D. N. Jansen, J. J. A. Keiren, A. J. Wijs. An $O(m \log n)$ Algorithm for Computing Stuttering Equivalence and Branching Bisimulation. *ACM Transactions on Computational Logic*, 18(2):13:1–13:34, 2017. (Versión extendida: arXiv:1909.10824, 2019.)
- [21] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.
- [22] N. D’Ippolito, D. Fischbein, M. Chechik, S. Uchitel. MTSA: The Modal Transition System Analyser. In *Proc. of the IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, pages 475–476, 2008. <http://mtsa.dc.uba.ar/>
- [23] mCRL2 toolset. Implementación de *branching bisimulation* de Groote y Jansen, archivo `liblts_bisim_gj.h`. https://github.com/mCRL2org/mCRL2/blob/master/libraries/lts/include/mcrl2/lts/detail/liblts_bisim_gj.h